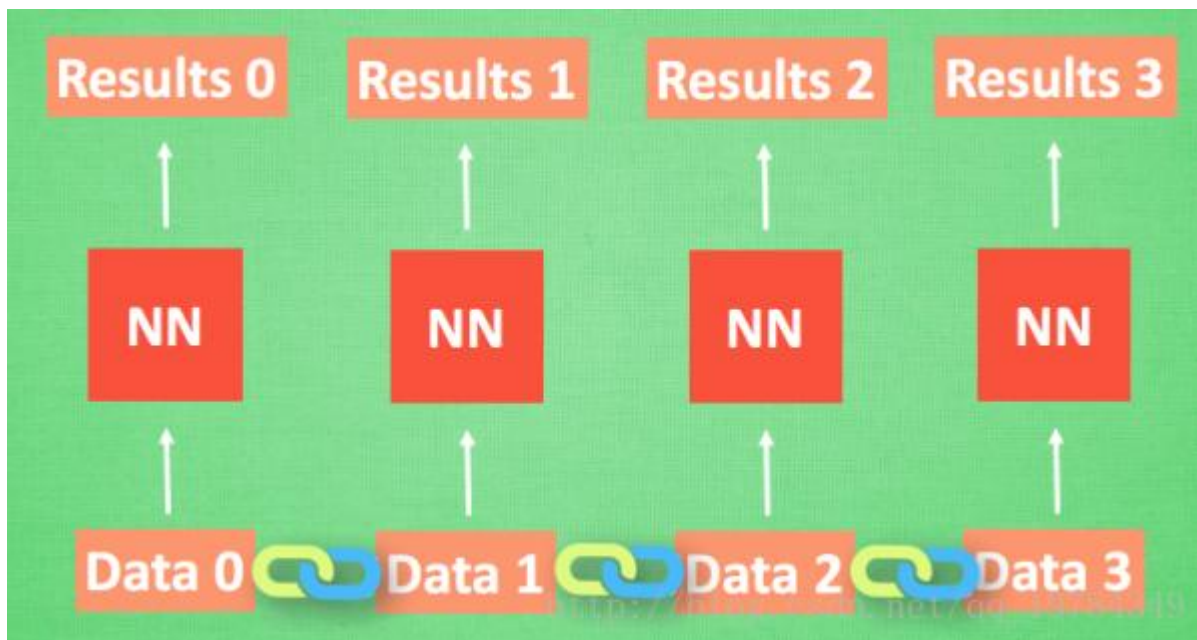
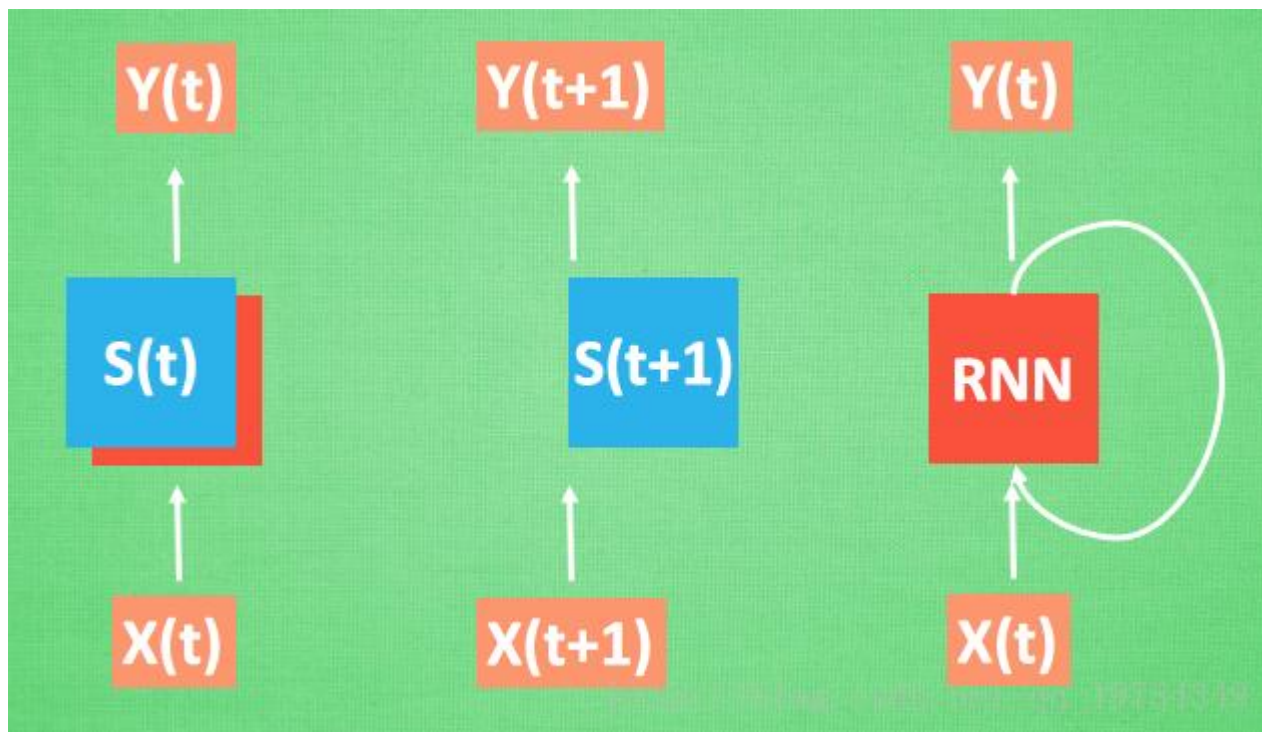


# 循环神经网络RNN



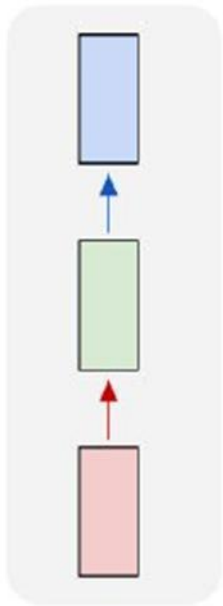
传统的NN都是离散地处理输入的序列信息，一个输入得到一个输出，彼此之间没有任何联系，这对于具有关联顺序的序列而言并不友好。



如果需要数据的关联性让神经网络知道，就需要让神经网络记住之前网络的信息，以达到网络信息的关联性。Data0 经过神经网络得到的当前状态描述 $S(t)$ ，由 $S(t)$ 生成 $Y(t)$ ；到了Data1 的时候，产生了 $S(t+1)$ ，但 $Y(t+1)$ 的生成是 $S(t)$ 和  $S(t+1)$  共同作用的结果。。

# 流水线经典结构神经网络

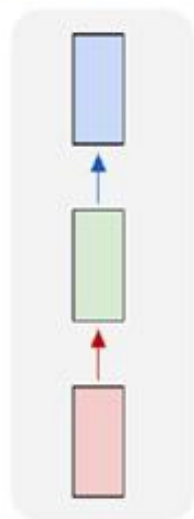
one to one



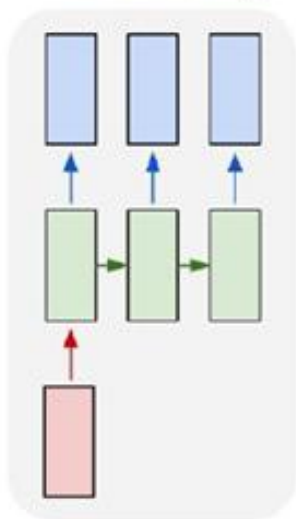
**Vanilla Neural  
Networks**

# “Vanilla” Neural Network

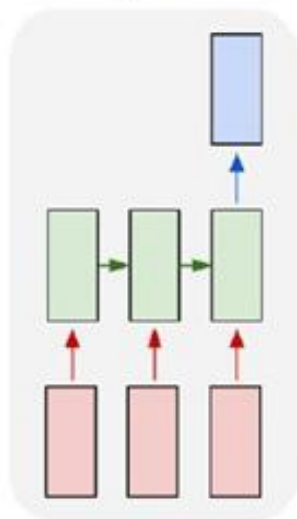
one to one



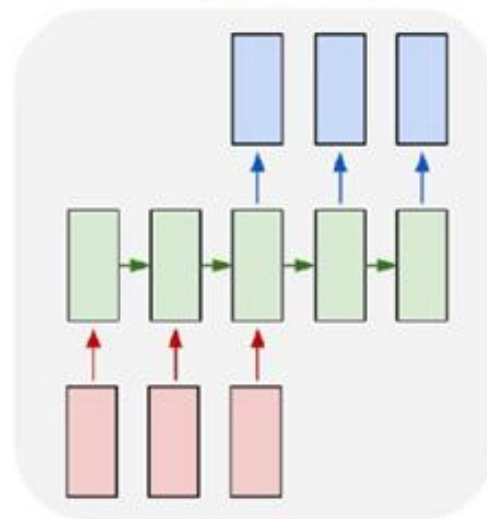
one to many



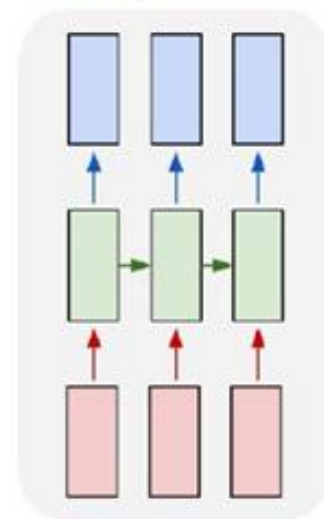
many to one



many to many



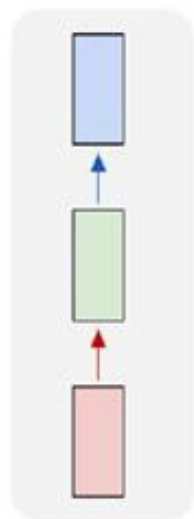
many to many



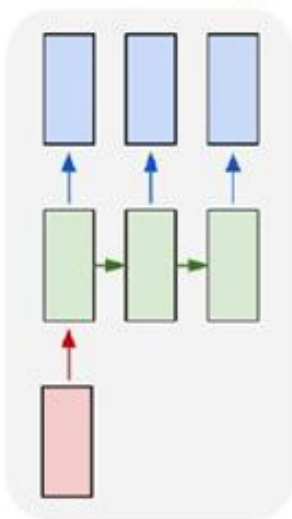
↖ e.g. 图像标题  
图像 -> 一句描述图片的话

# “Vanilla” Neural Network

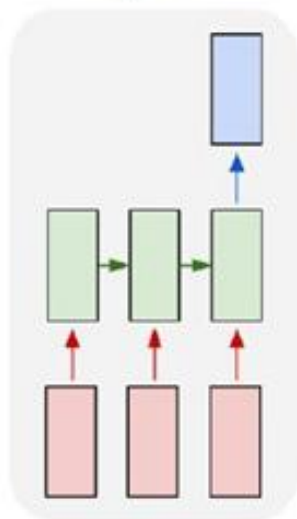
one to one



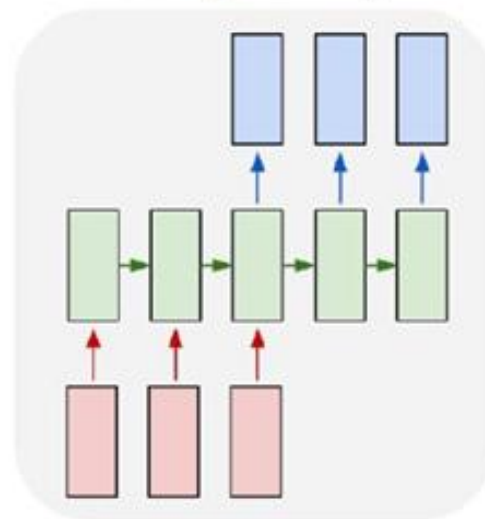
one to many



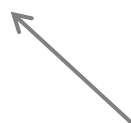
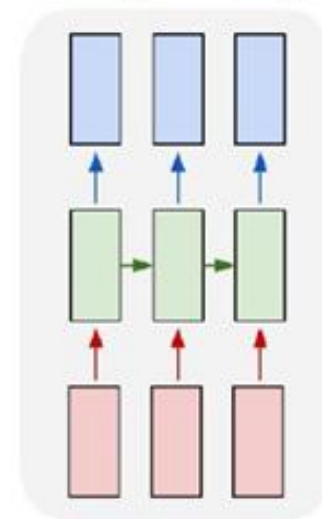
many to one



many to many



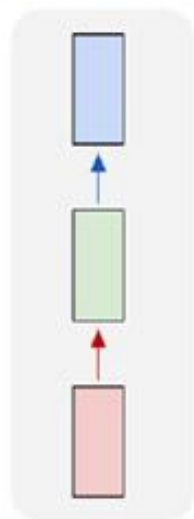
many to many



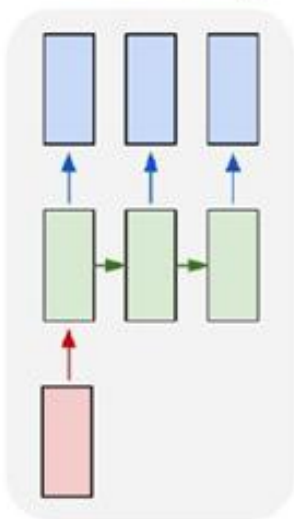
e.g. 情绪分类  
一句话 -> 高兴、悲伤

# “Vanilla” Neural Network

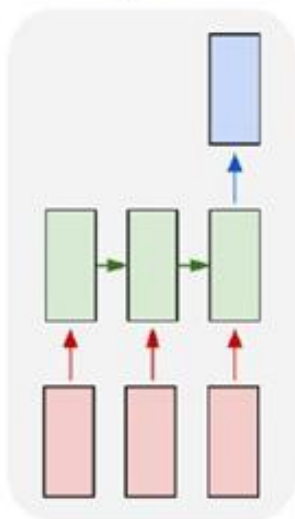
one to one



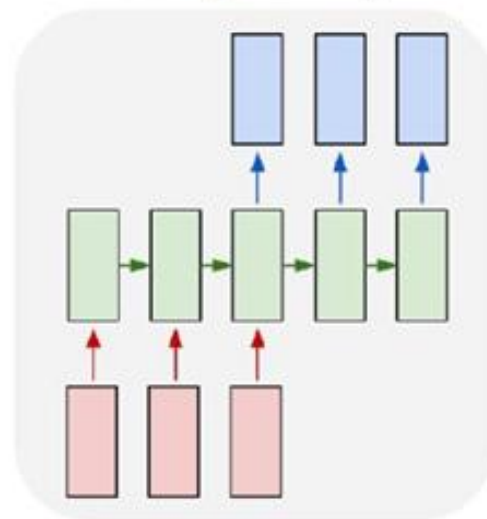
one to many



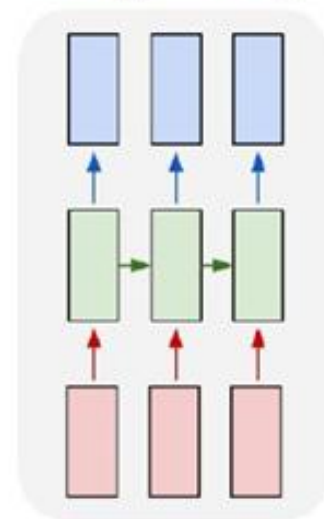
many to one



many to many



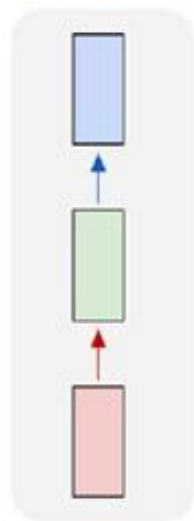
many to many



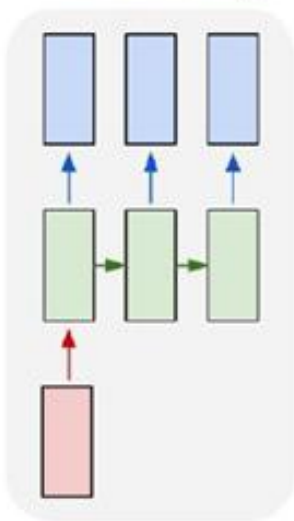
e.g. 机器翻译（有延时）  
词序-> 词序，不定长的

# “Vanilla” Neural Network

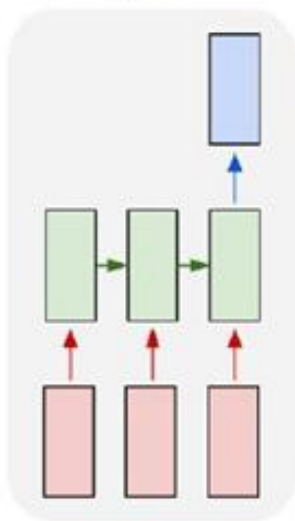
one to one



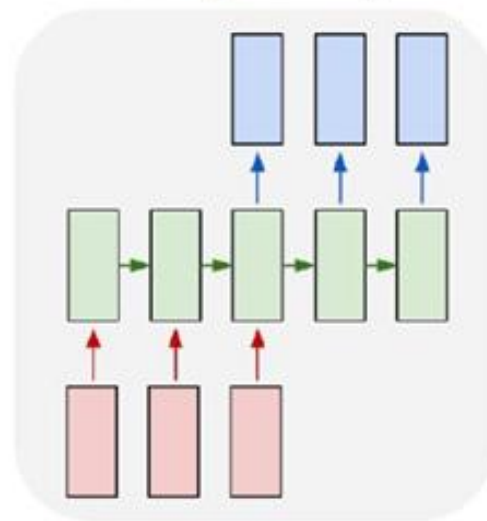
one to many



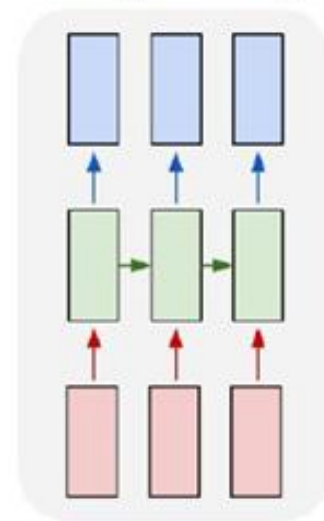
many to one



many to many



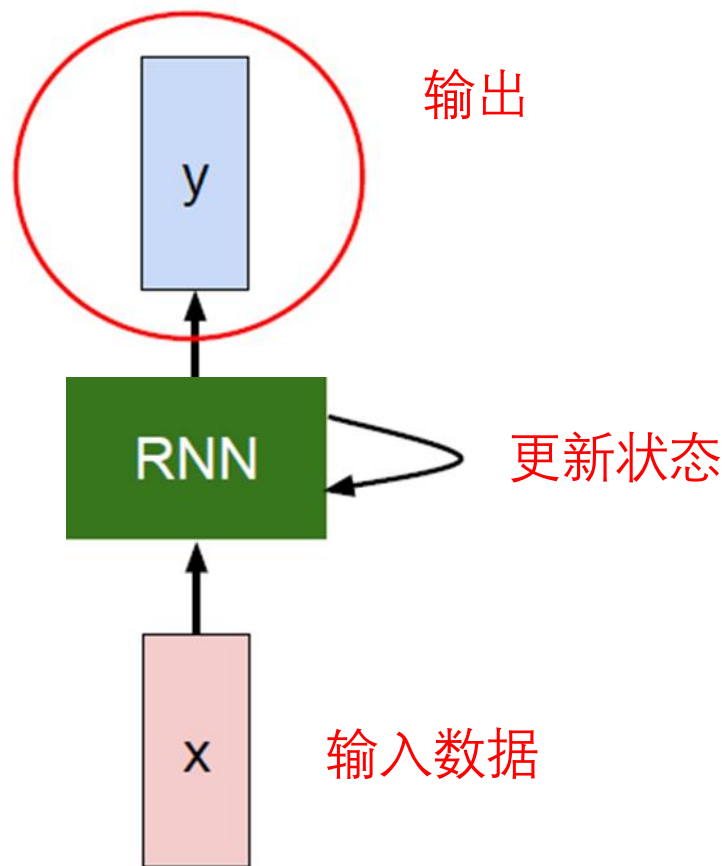
many to many



e.g. 视频的帧级分类，定长的



# 循环神经网络



**RNN通用架构：**每个RNN网络都有一个循环单元，输入，输出。通用架构以及循环函数如图所示，绿色框为循环单元。

**RNN大致工作流程：**首先， $x$ 传入循环单元，循环单元会生成一个状态 $h$ 。这个状态称为内部隐藏态(internal hidden state)。然后，当读取下一次输入时，这个状态会反馈到循环单元，生成新的状态。

**RNN模式：**读取输入，更新隐藏态，并生成输出。

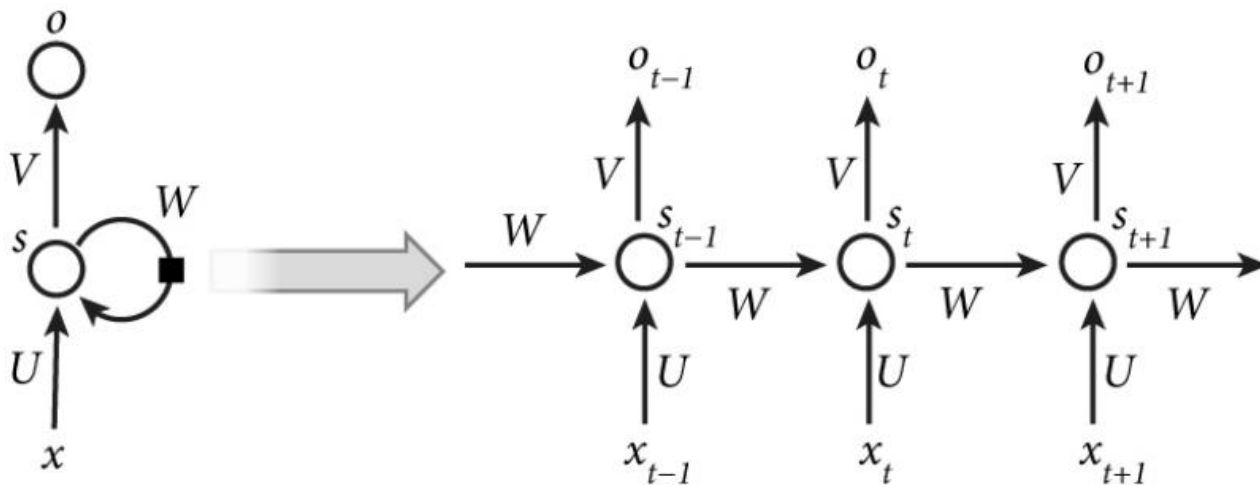
**循环函数：** $f(h, x)$ ，其中 $h$ 为状态， $x$ 为输入，这两个变量都是 $f$ 的输入， $f$ 输出新的状态。该函数含有权重 $W$ ，在循环时， $W$ 不变。

**如何在每一时间步给出输出 $y$ ：**在每一个时间步，将当时的状态 $h$ 输入到全连接层中，进行决策。

一个RNN的输入包括了两个：一个是当前时刻输入 $x_t$ ，用于实时更新状态，另一个是上一时刻隐藏层的状态 $h_{t-1}$ ，用于记忆状态，而不同时刻的网络共用的是同一套参数

# 循环神经网络

循环神经网络会记忆之前的信息，并利用之前的信息影响后面节点的输出。也就是说，循环神经网络的隐藏层之间的节点是有连接的，隐藏层的输入不仅包括输入层的输出，还包括上一时刻隐藏层的输出。



# 循环神经网络

我们可以通过在每个时间步骤应用递推公式来处理向量  $x$  的序列：

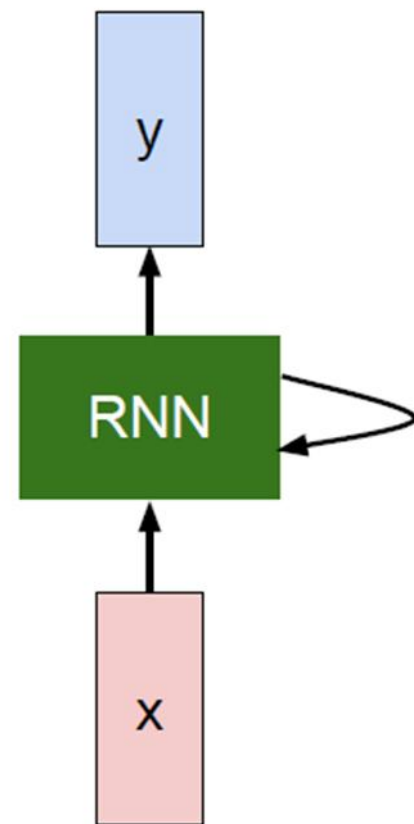
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

新状态

参数  $W$  的一些函数

旧状态

在某个时间步输入矢量



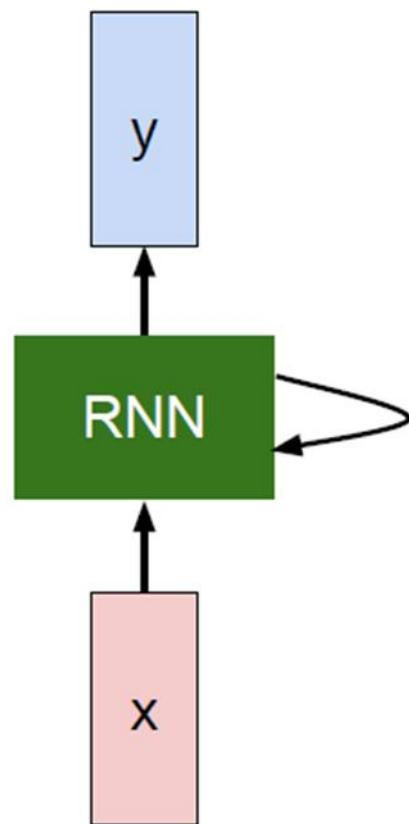
每个时刻，函数  $f$  接收上一时刻的隐藏态  $h_{t-1}$  和当前时刻的输入  $x_t$ ，产生当前时刻的隐藏态  $h_t$ 。其中函数  $f$  是关于参数  $w$  的函数，输出  $y$  可以由当前的隐藏态产生。

# 循环神经网络

我们可以通过在每个时间步骤应用递推公式来处理向量x的序列：

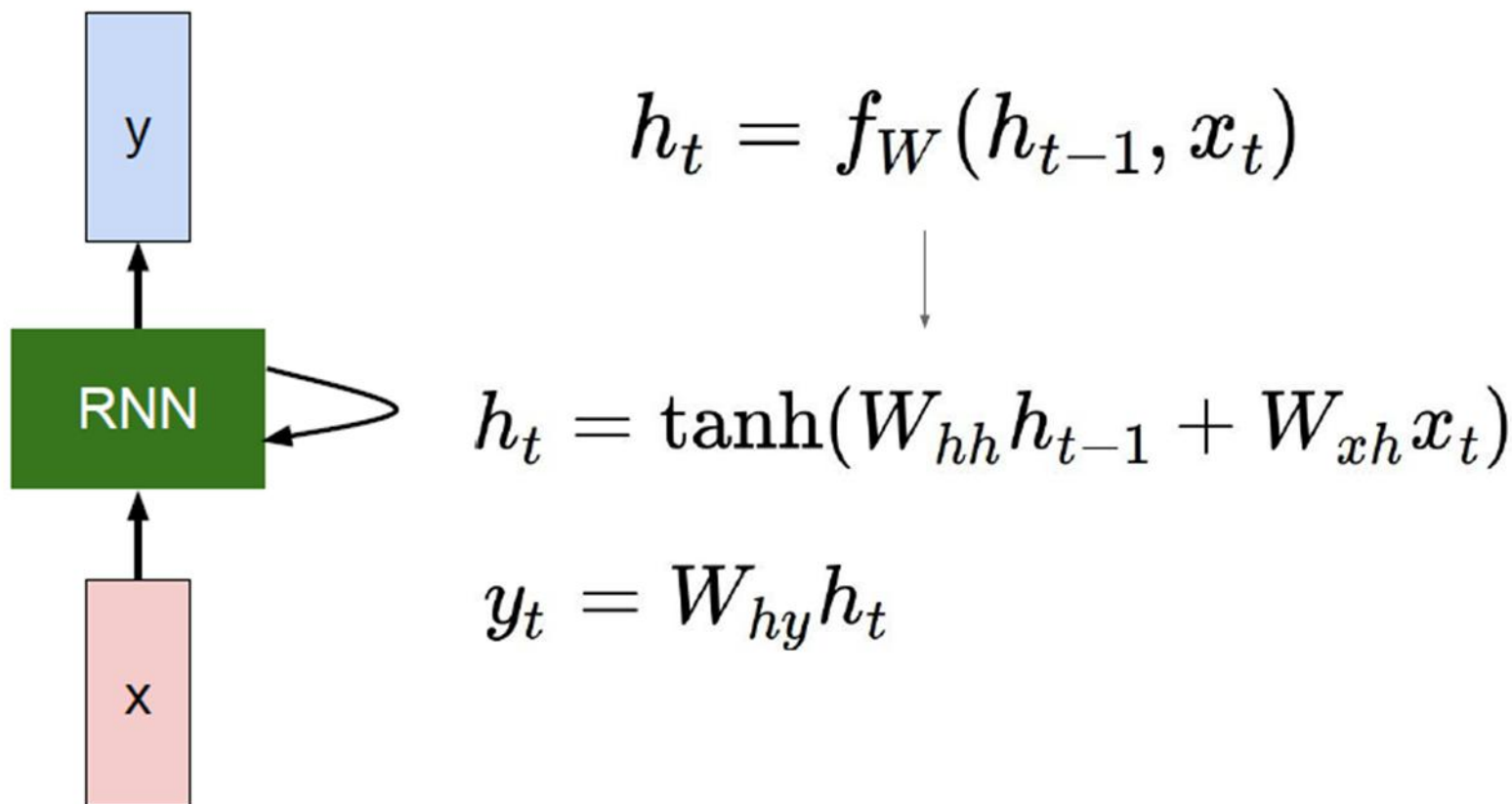
$$h_t = f_W(h_{t-1}, x_t)$$

注意：在每个时间步使用相同的功能和相同的一组参数。



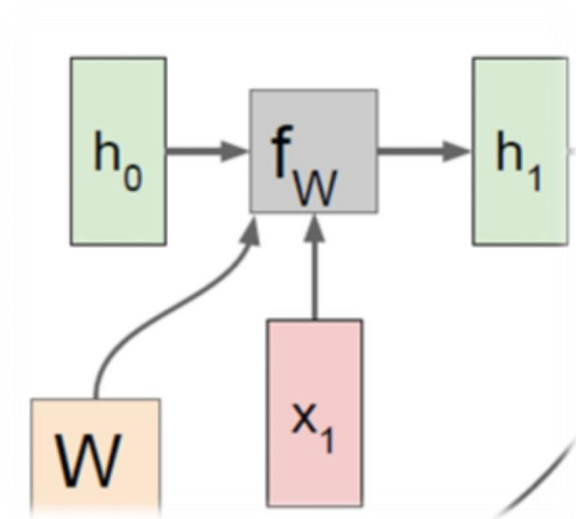
# (Vanilla) Recurrent Neural Network

状态由一个“隐藏”向量 $h$ 组成：



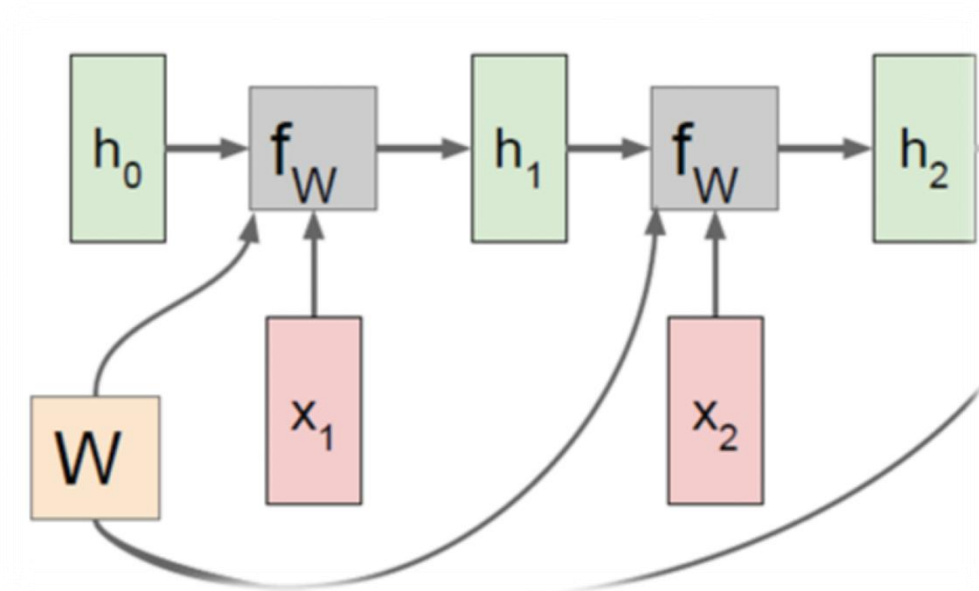
# 几种RNN计算图

在每个时间步重复使用相同的权重矩阵



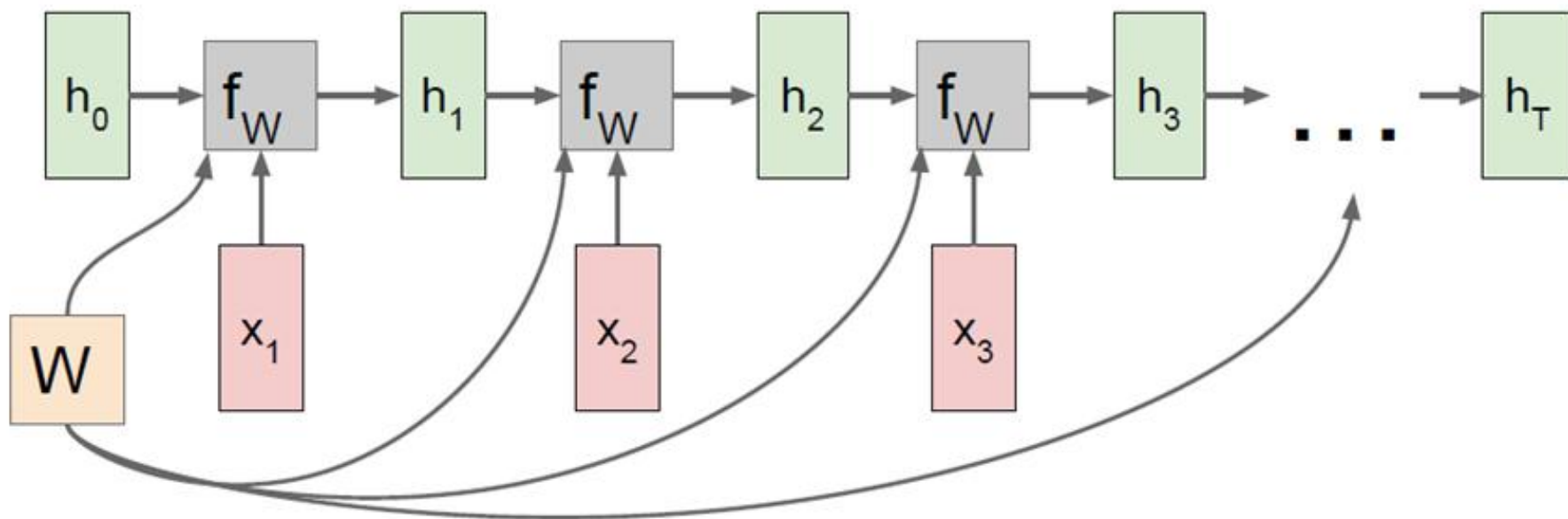
## RNN: 计算图

在每个时间步重复使用相同的权重矩阵



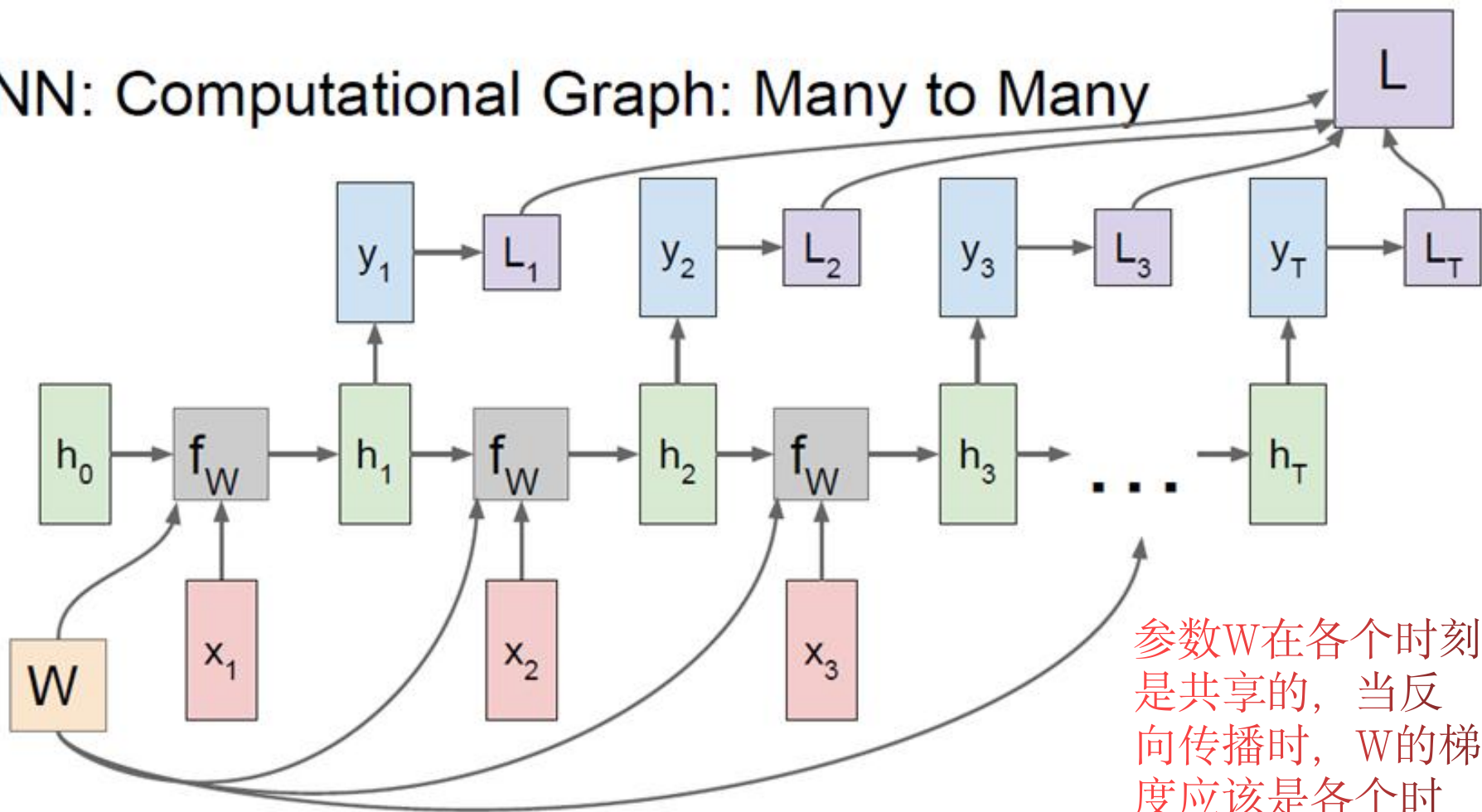
## RNN: 计算图

在每个时间步重复使用相同的权重矩阵



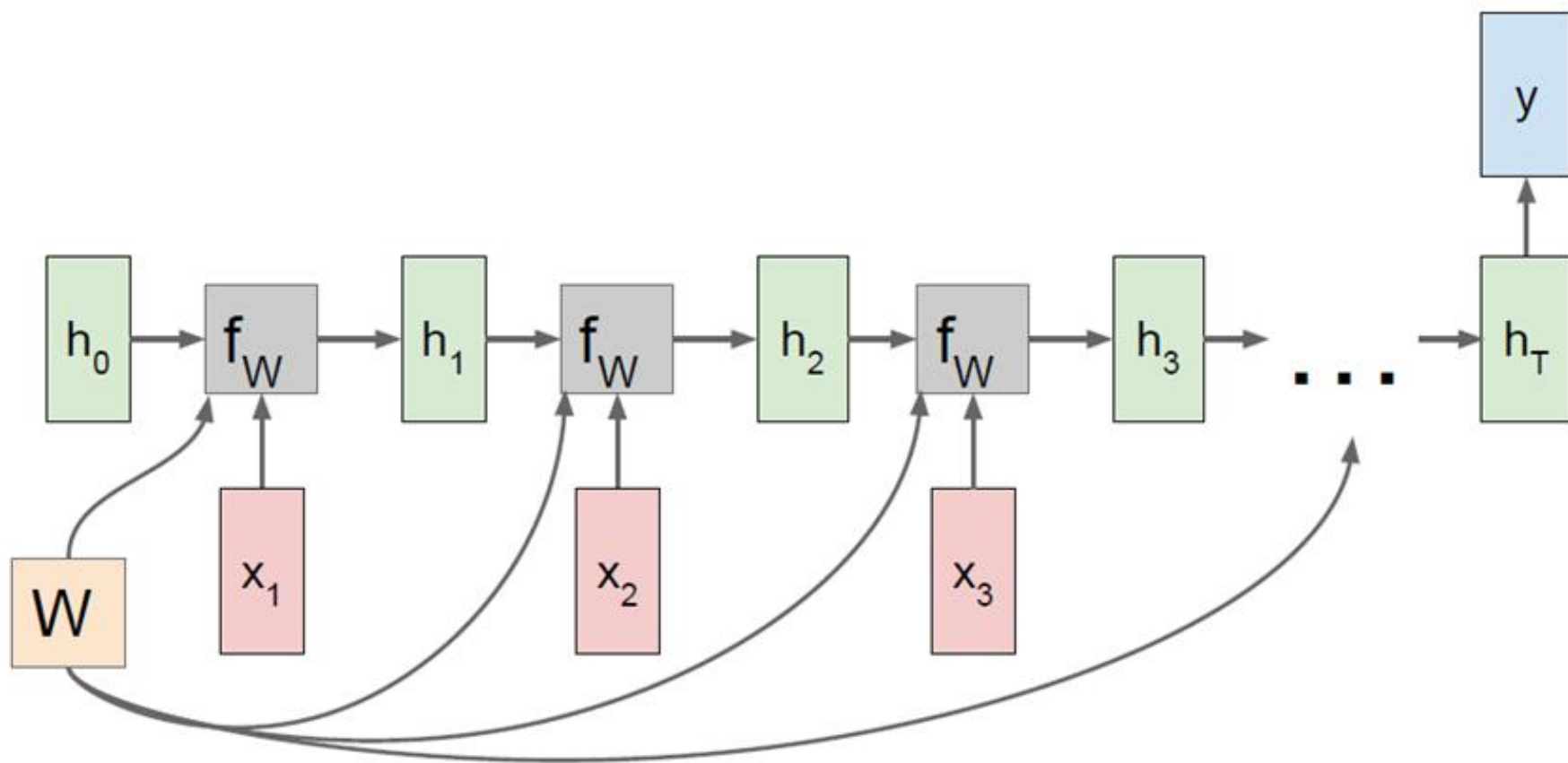


## RNN: Computational Graph: Many to Many

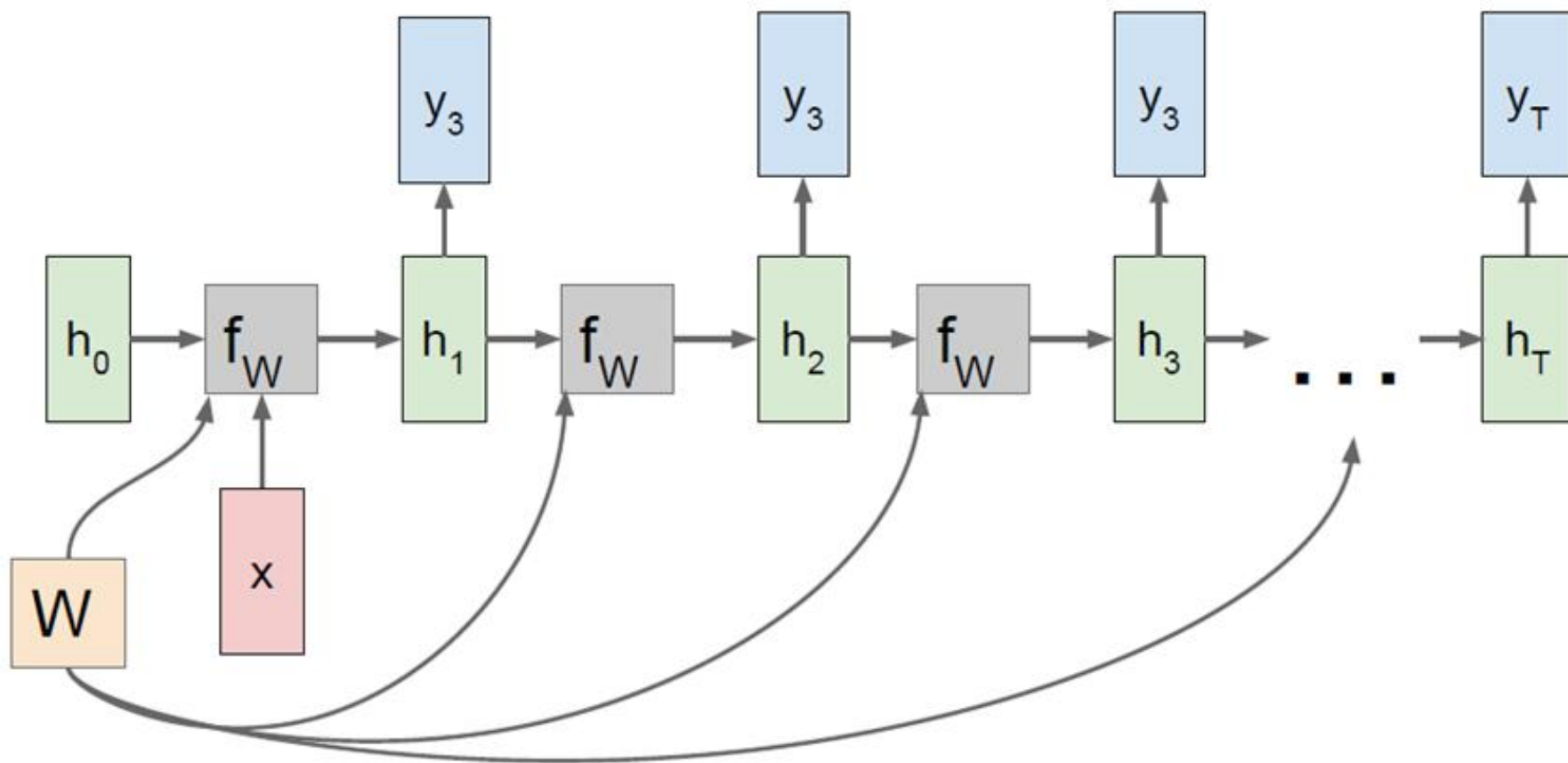


参数 $W$ 在各个时刻是共享的，当反向传播时， $W$ 的梯度应该是各个时刻传过来的梯度之和。

## RNN计算图：Many to One



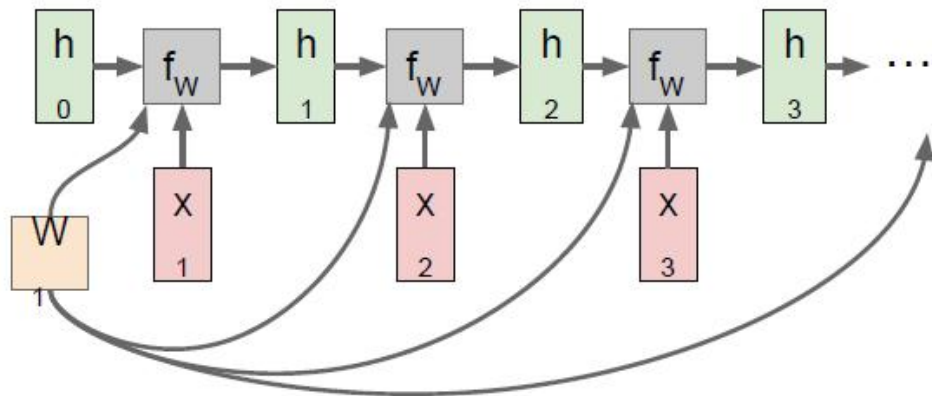
## RNN计算图：One to Many



# 序列到序列：Many-to-one + one-to-many

seq2seq的计算图，可以被拆解为many2one和one2many两部分。其中many2one部分被称为编码器，将输入x编码成一个定长的向量。one2many被称为解码器，负责将该向量转化成输出序列

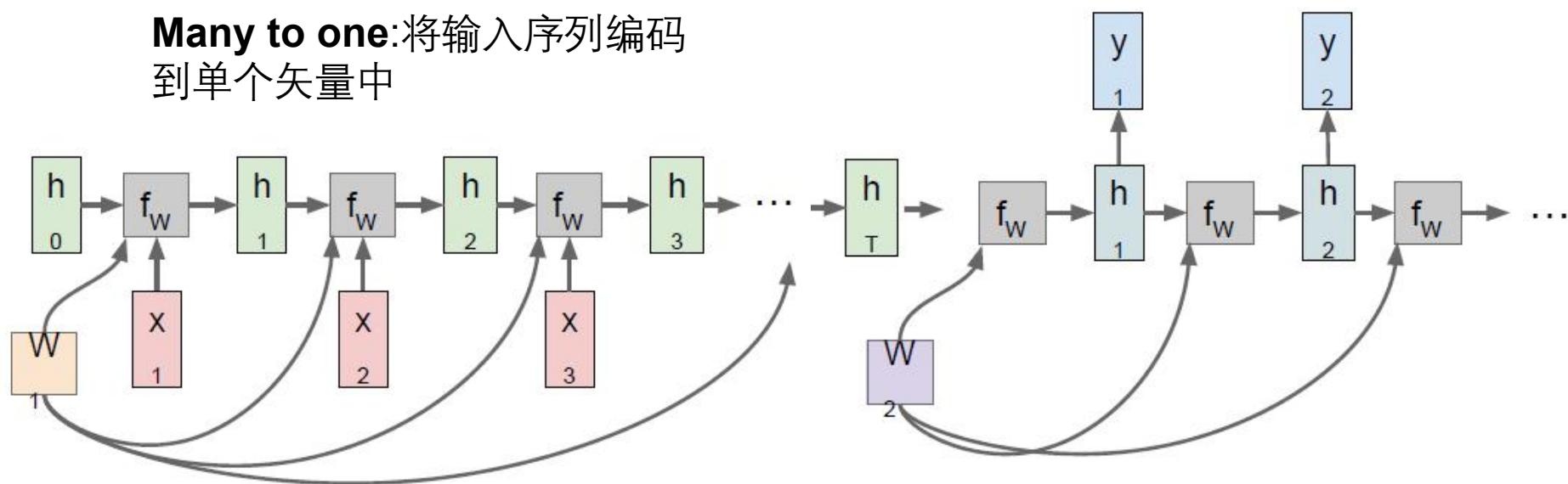
**编码 Many to one:**将输入序列  
编码到单个矢量中



# 序列到序列: Many-to-one + one-to-many

**解码One to many:**从单个输入向量生成输出序列

**Many to one:**将输入序列编码到单个矢量中



# RNN示例

模型训练

字符级语言模型

预测下一个出现字母

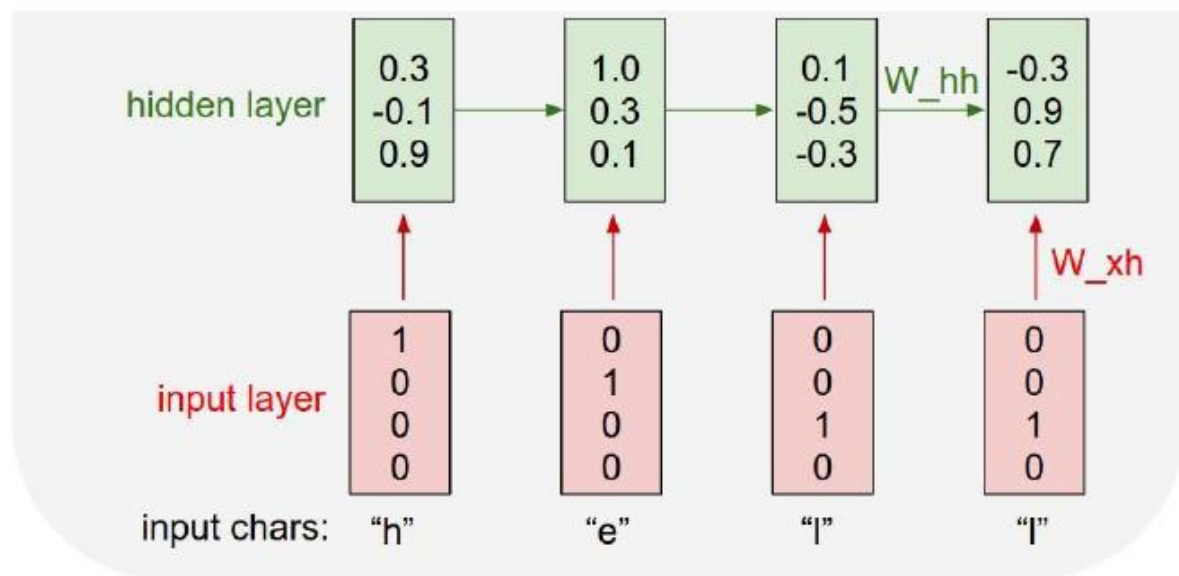
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

词汇:

[h,e,l,o]

训练样本序列:

“hello”



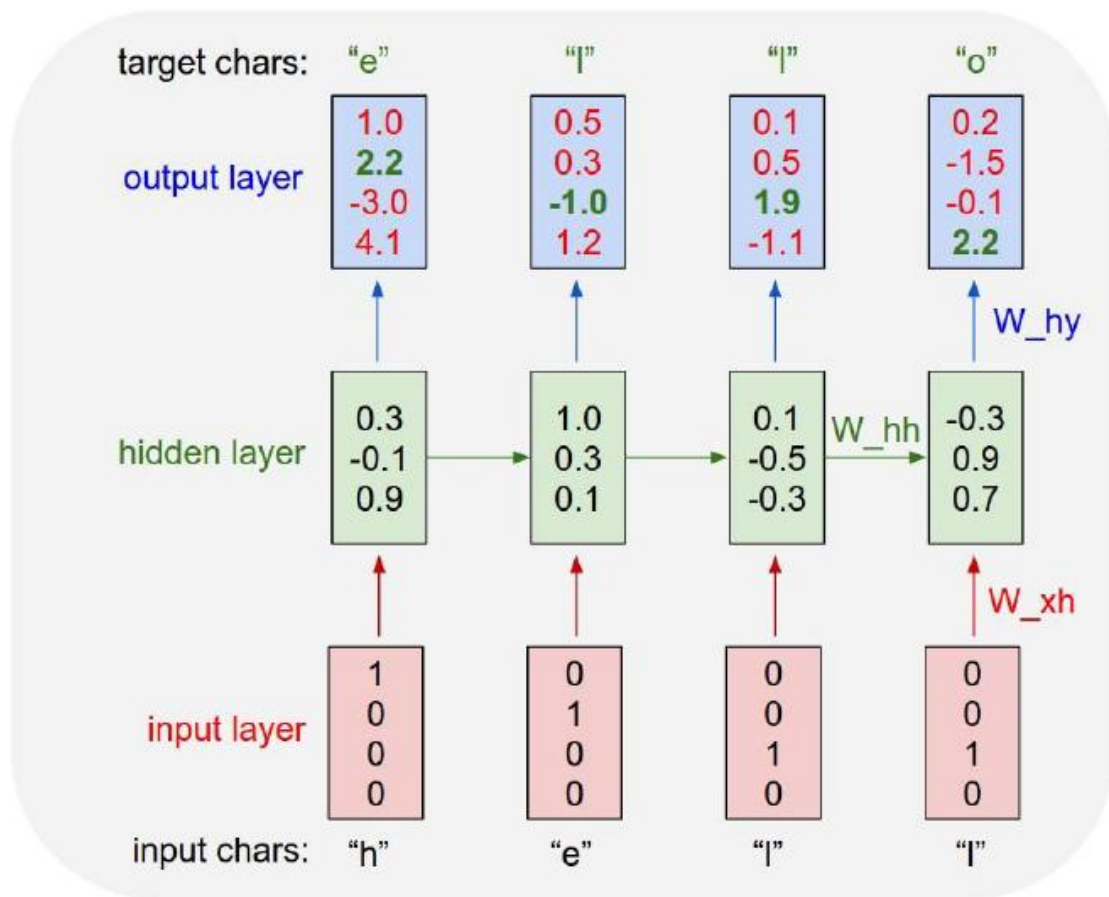
# 模型训练

## 字符级

## 语言模型

词汇:  
[h,e,l,o]

训练样本序列:  
“hello”

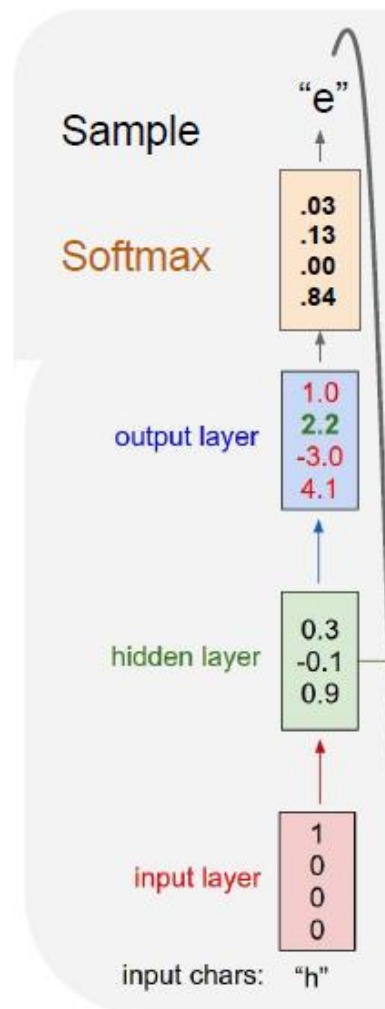


训练阶段，输入一个样本“hello”，我们经过一次前向传播产生输出。而我们的目标输出应该是输入序列向右平移一格，据此计算出损失，进而反向传播计算梯度，更新参数值，完成一轮迭代，进入下一次迭代。

# 模型测试： softmax采样

词汇：  
[h,e,l,o]

在测试时间一个字符  
的样本，反馈到模型

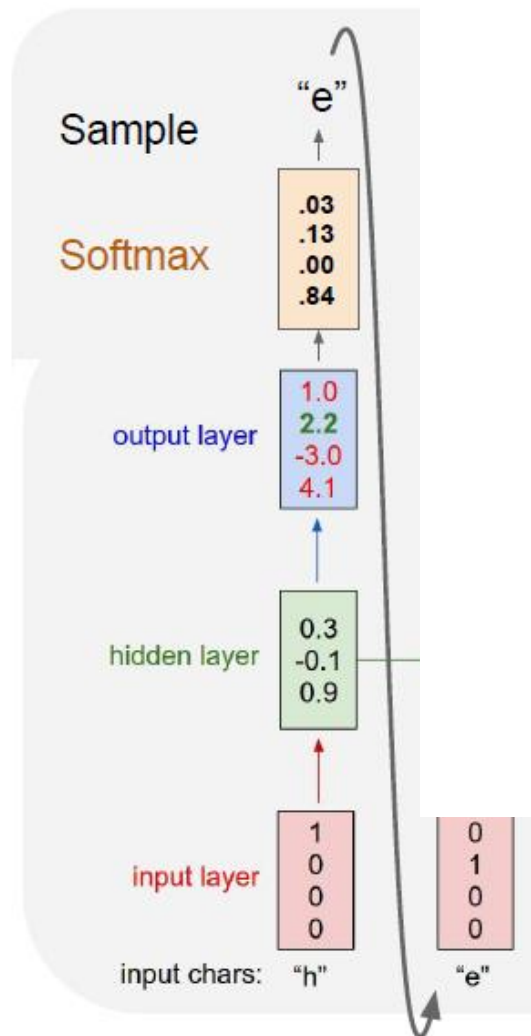




# 模型测试： 字符级 语言模型 采样

词汇：  
[h,e,l,o]

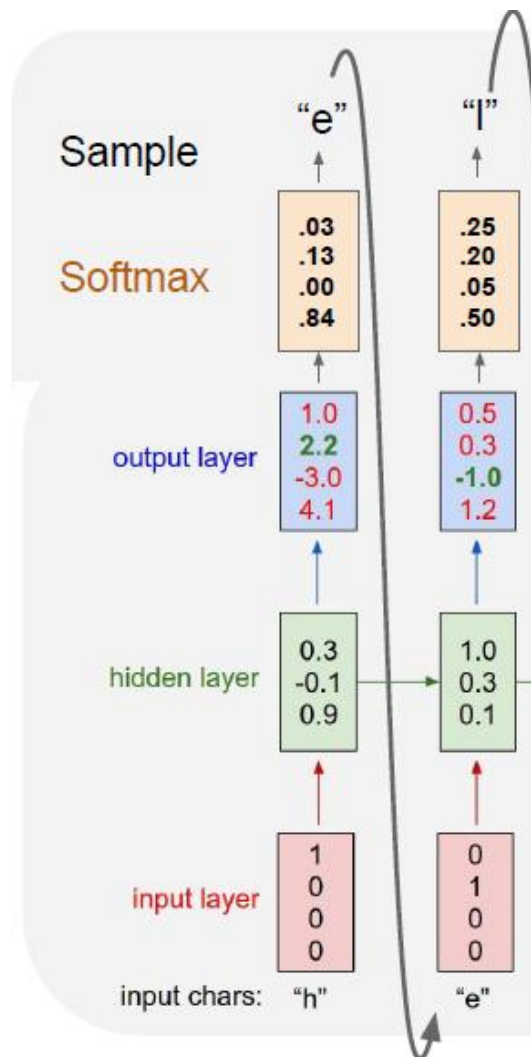
在测试时间一个字符  
的样本，反馈到模型



# 字符级 语言模型 采样

词汇:  
[h,e,l,o]

在测试时间一个字符  
的样本，反馈到模型



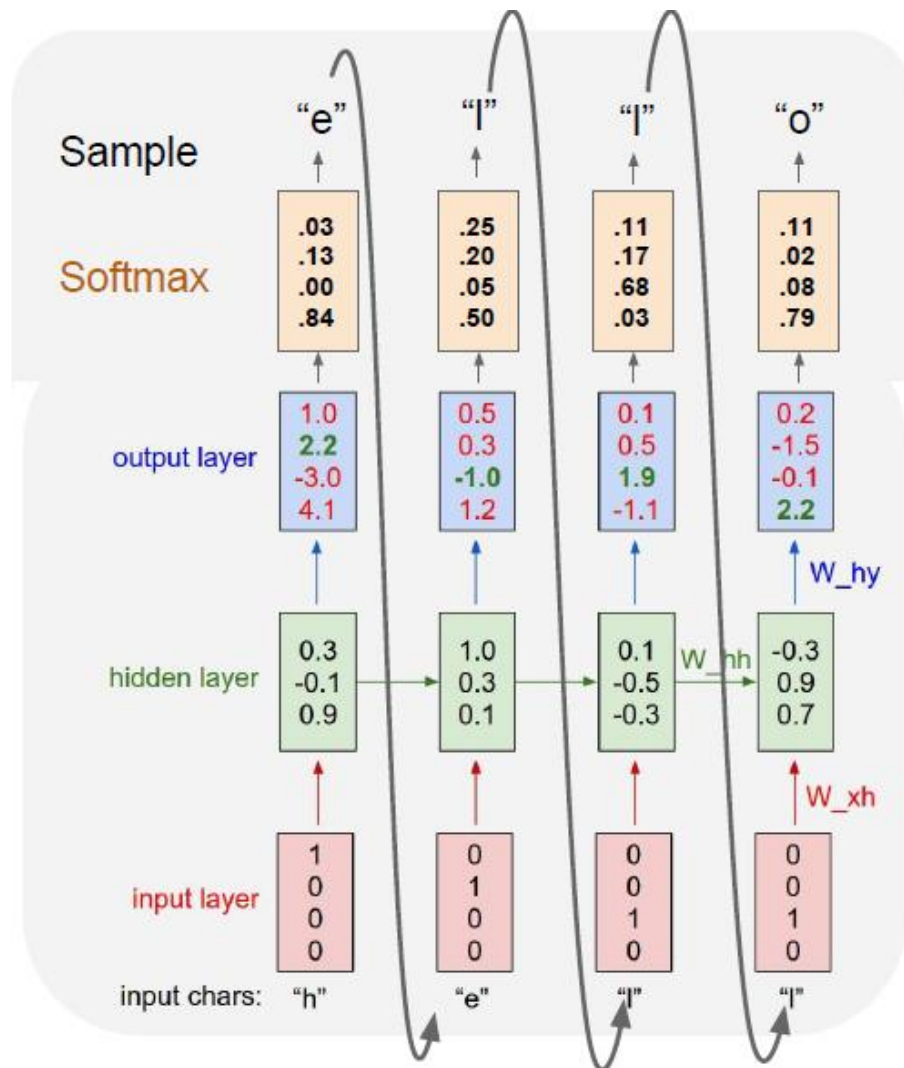
Example:

# 字符级 语言模型 采样

词汇:

[h,e,l,o]

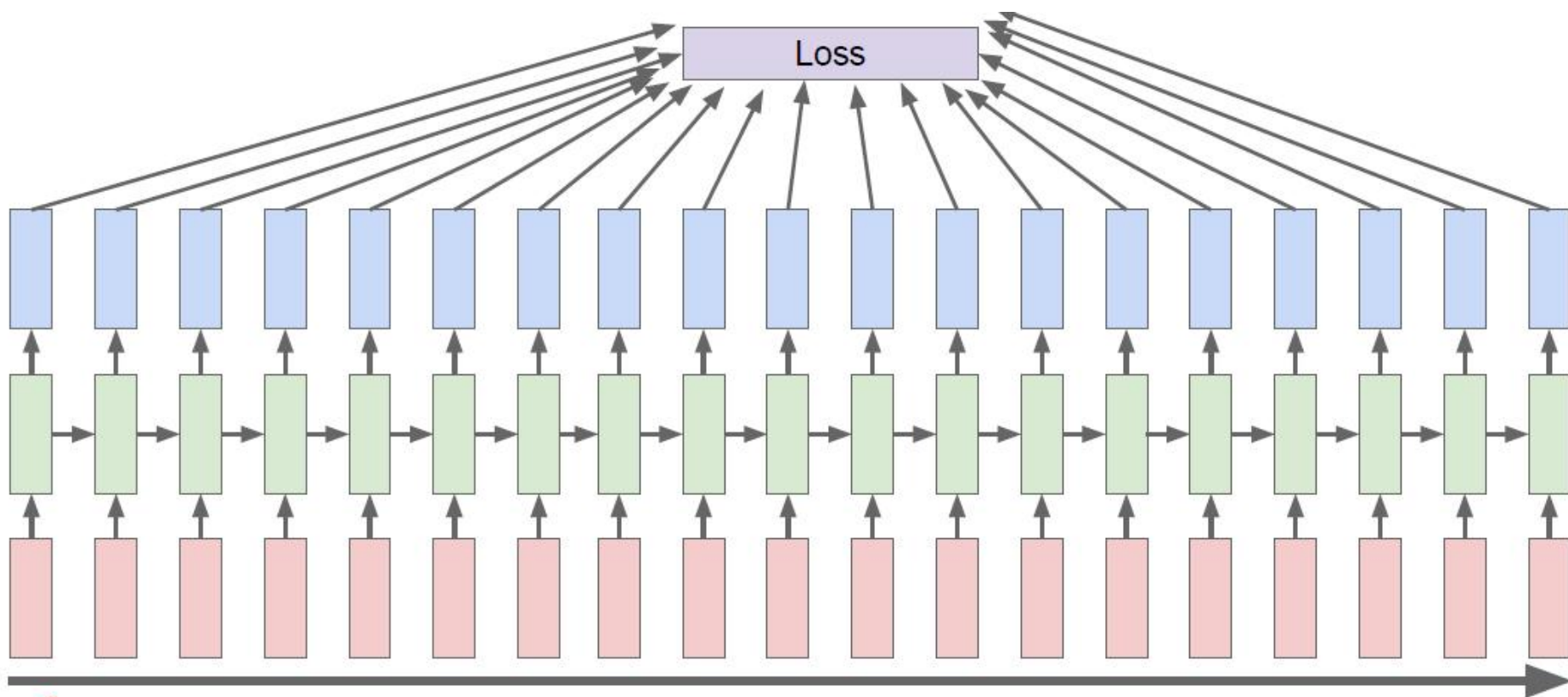
在测试时间一个字符  
的样本，反馈到模型



# 依序反向传播

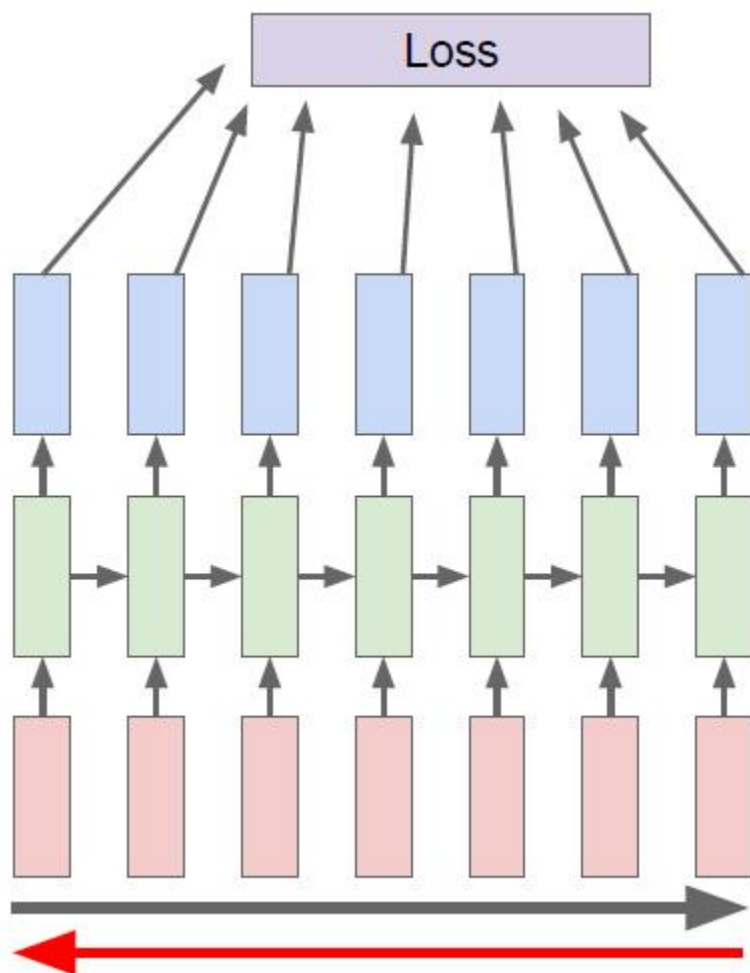
大数据时计算缓慢

1. 计算前馈通道获得Loss,
2. 向后遍历整个序列计算梯度



因为每次loss的计算，梯度计算和BP依赖于模型输入序列的长度。当输入序列很长时，BP就十分耗时。模型很难收敛，并且占用非常大的内存。

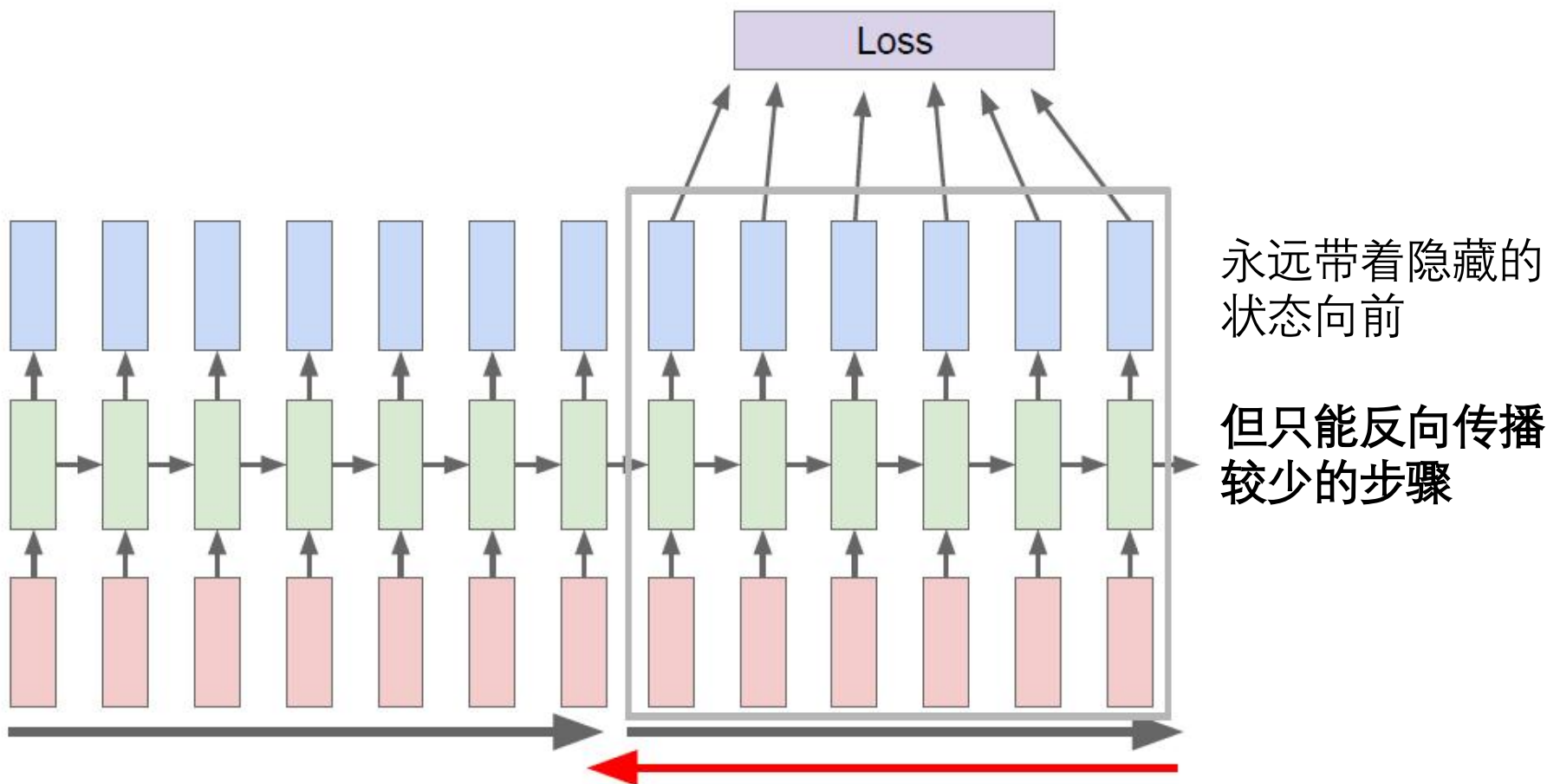
# 截断依序反向传播（估计）



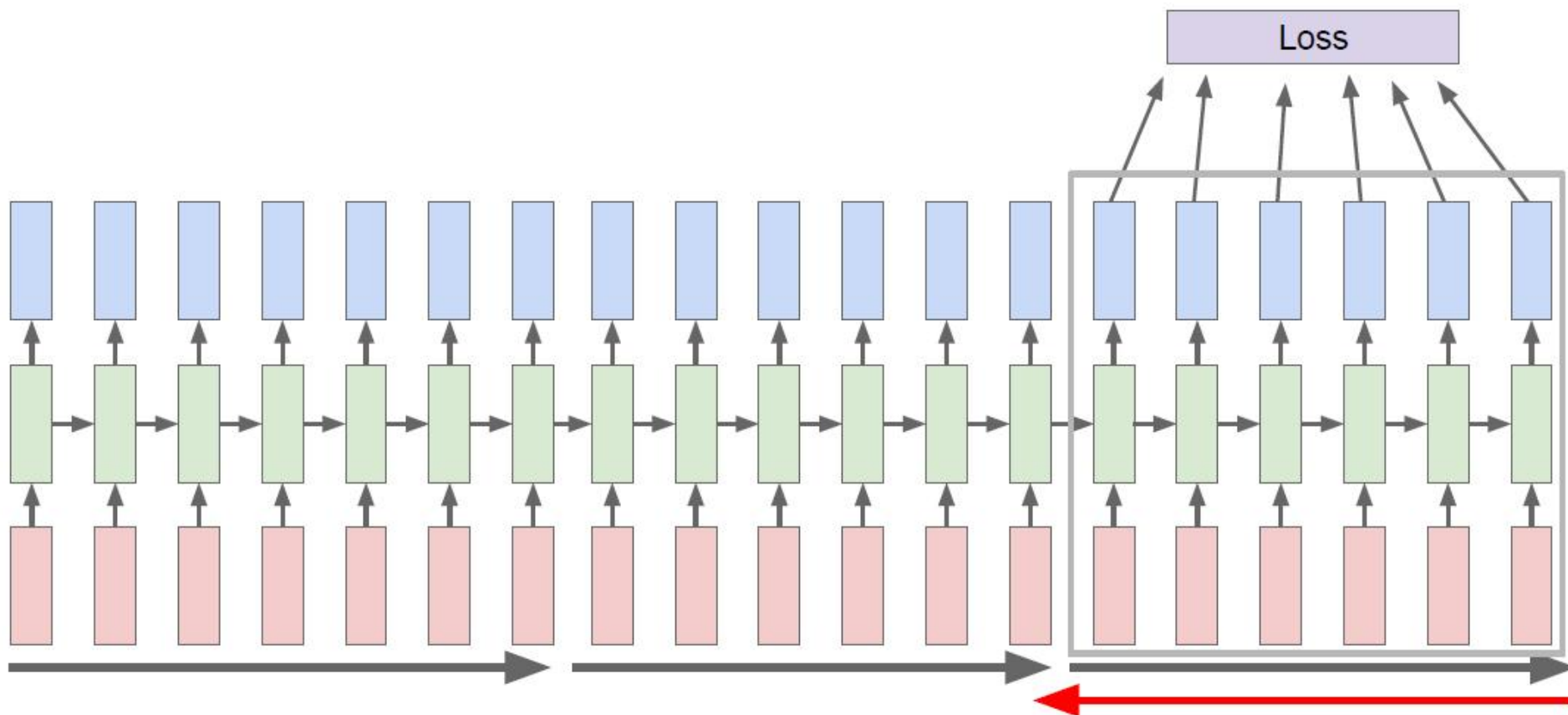
通过序列的块而不是整个序列向前和向后运行

沿时间的截断反向传播方法。每 $n$ 个时间步，就计算一次损失以及梯度，在前 $n$ 个时间步内进行BP，更新权重。min-char-rnn.py实现了该方法

# 截断依序反向传播（估计）



## 截断依序反向传播（估计）



# min-char-rnn.py gist: 112 lines of Python

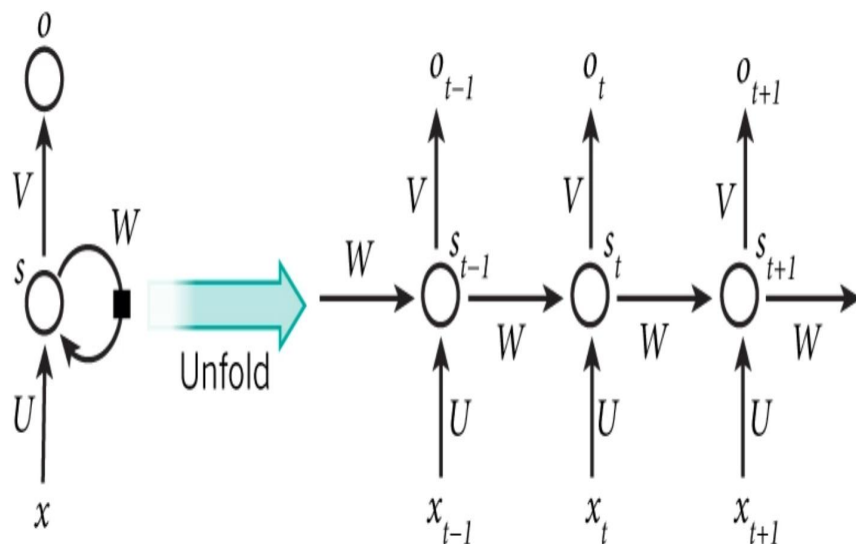
```
1 """
2 Minimal character-level vanilla RNN model. Written by Andre Karpathy (@karpathy)
3 BSD license
4 """
5 import numpy as np
6
7 # DATA I/O
8 data = open('input.txt', 'r').read() # should be simple plain text file
9 chars = list(set(data))
10 data_size, vocab_size = len(data), len(chars)
11 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
12 char_to_ix = { ch:i for i,ch in enumerate(chars) }
13 ix_to_char = { i:ch for i,ch in enumerate(chars) }
14
15 # Hyperparameters
16 hidden_size = 100 # size of hidden layer of neurons
17 seq_length = 25 # number of steps to unroll the RNN for
18 learning_rate = 1e-1
19
20 # Model parameters
21 wh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
22 whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
23 why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
24 bh = np.zeros((hidden_size, 1)) # hidden bias
25 by = np.zeros((vocab_size, 1)) # output bias
26
27 def lossfun(inputs, targets, hprev):
28     """
29     inputs, targets are both list of integers.
30     hprev is Numpy array of initial hidden state
31     returns the loss, gradients on model parameters, and last hidden state
32     """
33     xs, hs, ys, ps = [], [], [], []
34     h = hprev
35     loss = 0
36     # Forward pass
37     for t in xrange(len(inputs)):
38         xs[t] = np.zeros((vocab_size, 1)) # encode in 1-of-k representation
39         xs[t][inputs[t]] = 1
40         hs[t] = np.tanh(np.dot(wh, xs[t]) + np.dot(whh, hs[t-1]) + bh) # hidden state
41         ys[t] = np.dot(why, hs[t]) + by # unnormalized log probabilities for next char
42         ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next char
43         loss += -np.log(ps[t][targets[t], 0]) # softmax (cross-entropy) loss
44     # Backward pass: compute gradients going backwards
45     dwh, dwhh, dwhy = np.zeros_like(wh), np.zeros_like(whh), np.zeros_like(why)
46     dbh, dby = np.zeros_like(bh), np.zeros_like(by)
47     dhnext = np.zeros_like(hs[-1])
48     for t in reversed(xrange(len(inputs))):
49         dy = np.copy(ps[t])
50         dy[targets[t]] -= 1 # backprop into y
51         dwhy += np.dot(dy, hs[t].T)
52         dby += dy
53         dh = np.dot(why.T, dy) + dhnext # backprop into h
54         draw = (1 - hs[t]) * hs[t] * dh # backprop through tanh nonlinearity
55         dwh += draw
56         dwhh += draw
57         dhnext = np.dot(whh.T, draw)
58     for dparam in [dwh, dwhh, dwhy, dbh, dby]:
59         np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
60     return loss, dwh, dwhh, dwhy, dbh, dby, hs[len(inputs)-1]
61
62 def sample(h, seed_ix, n):
63     """
64     sample a sequence of integers from the model
65     h is memory state, seed_ix is seed letter for first time step
66     """
67     x = np.zeros((vocab_size, 1))
68     s[seed_ix] = 1
69     ixes = []
70     for t in xrange(n):
71         h = np.tanh(np.dot(wh, x) + np.dot(whh, h) + bh)
72         y = np.dot(why, h) + by
73         p = np.exp(y) / np.sum(np.exp(y))
74         ix = np.random.choice(range(vocab_size), p=p.ravel())
75         x = np.zeros((vocab_size, 1))
76         x[ix] = 1
77         ixes.append(ix)
78     return ixes
79
80 #, p = 0, 0
81 dwh, dwhh, dwhy = np.zeros_like(wh), np.zeros_like(whh), np.zeros_like(why)
82 dbh, dby = np.zeros_like(bh), np.zeros_like(by) # memory variables for adagrad
83 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
84 while True:
85     # prepare inputs (we're unrolling from left to right in steps seq_length long)
86     if p+seq_length-1 == len(data) or p == 0:
87         hprev = np.zeros((hidden_size,1)) # reset rnn memory
88         p = 0 # go from start of data
89     inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
90     targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
91
92     # sample from the model now and then
93     if n % 100 == 0:
94         sample_ix = sample(hprev, inputs[0], 200)
95         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
96         print '....\n %s \n....' % (txt, )
97
98     # Forward seq_length characters through the net and fetch gradient
99     loss, dwh, dwhh, dwhy, dbh, dby, hprev = lossfun(inputs, targets, hprev)
100     smooth_loss = smooth_loss * 0.999 + loss * 0.001
101     if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
102
103     # perform parameter update with adagrad
104     for param, dparam, mem in zip([wh, whh, why, bh, by],
105                                   [dwh, dwhh, dwhy, dbh, dby],
106                                   [dwh, dwhh, dwhy, dbh, dby]):
107         mem += dparam * dparam
108         param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
109
110     p += seq_length # move data pointer
111     n += 1 # iterations counter
112
```

(<https://gist.github.com/karpathy/d4dee566867f8291f086>)



# BPTT (backpropagation through time)

$x_t$  是当前时间步的输入,  $s_t$  是当前的状态,  $\hat{y}_t$  是输出



$$s_t = \tanh(Ux_t + Ws_{t-1})$$

$$\hat{y}_t = \text{softmax}(Vs_t)$$

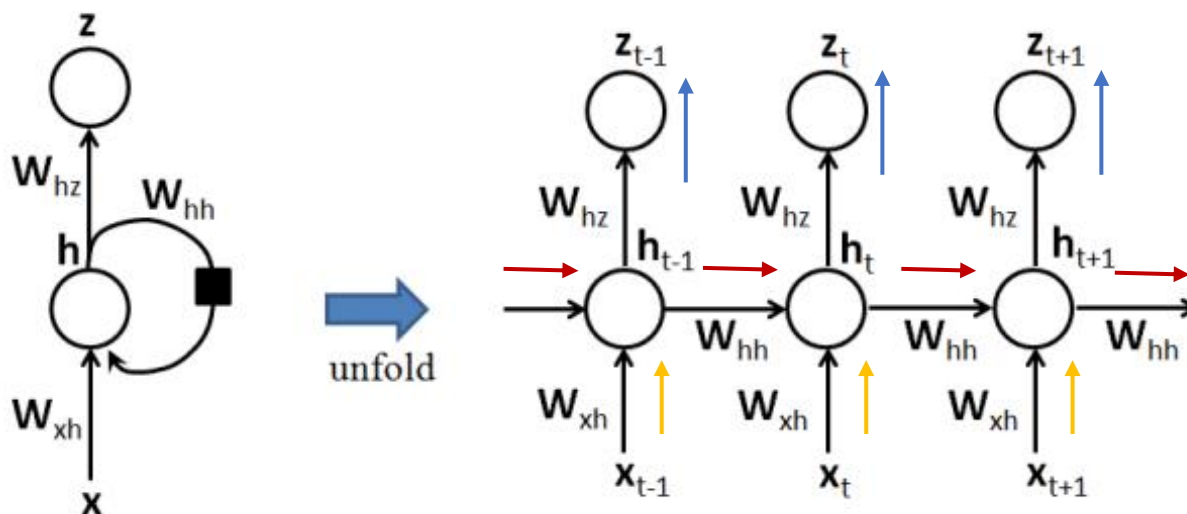
交叉熵作为代价函数

$$\begin{aligned} E_t(y_t, \hat{y}_t) &= -y_t \log \hat{y}_t \\ E(y, \hat{y}) &= \sum_t E_t(y_t, \hat{y}_t) \\ &= - \sum_t y_t \log \hat{y}_t \end{aligned}$$

为了体现RNN的循环性, 可以将多层fold起来, 每个部分看做一个cell, 处理过程一模一样,

# BPTT (backpropagation through time)

时序扩展



$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{z}_t = \text{softmax}(\mathbf{W}_{hz}\mathbf{h}_t + \mathbf{b}_z)$$

# BPTT (backpropagation through time)

BP回顾：定义损失函数  $E$  来表示输出  $\hat{y}$  和真实标签  $y$  的误差，通过链式法则自顶向下求得  $E$  对网络权重的偏导。沿梯度的反方向更新权重的值，直到  $E$  收敛。

BPTT的本质其实和BP很像，就是加上了时序演化。定义权重  $U$ ,  $V$ ,  $W$ 。

$$s_t = \tanh(Ux_t + Ws_{t-1})$$

$$\hat{y}_t = \text{softmax}(Vs_t)$$

定义损失函数：  $E_t(y_t, \hat{y}_t) = -y_t \log \hat{y}_t$

$$\begin{aligned} E(y, \hat{y}) &= \sum_t E_t(y_t, \hat{y}_t) \\ &= - \sum_t y_t \log \hat{y}_t \end{aligned}$$

我们将整个序列作为一次训练，所以需要每个时刻的误差进行求和。

# BPTT (backpropagation through time)

目前的任务是求  $E$  对于  $U$ ,  $V$ ,  $W$  的梯度。  
定义  $E$  对于  $W$  的梯度 ( $U$ ,  $V$  同理):

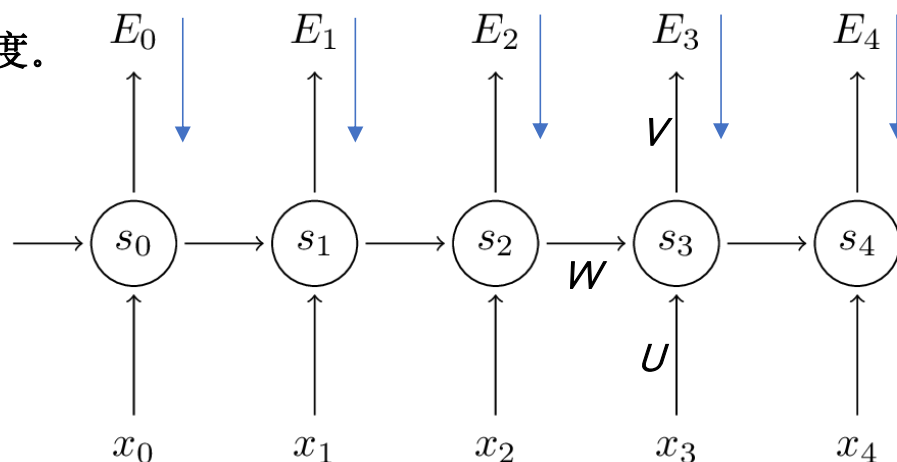
$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

(1) 求  $E$  对于  $V$  的梯度。

先求  $E_3$  对于  $V$  的梯度:

$$\begin{aligned} \frac{\partial E_3}{\partial V} &= \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial V} \\ &= \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial z_3}{\partial V} \end{aligned}$$

其中:  $z_3 = V s_3$



$$\frac{\partial E}{\partial V} \quad \text{求和可得。}$$

# BPTT (backpropagation through time)

$\frac{\partial E}{\partial W}$  求和可得。

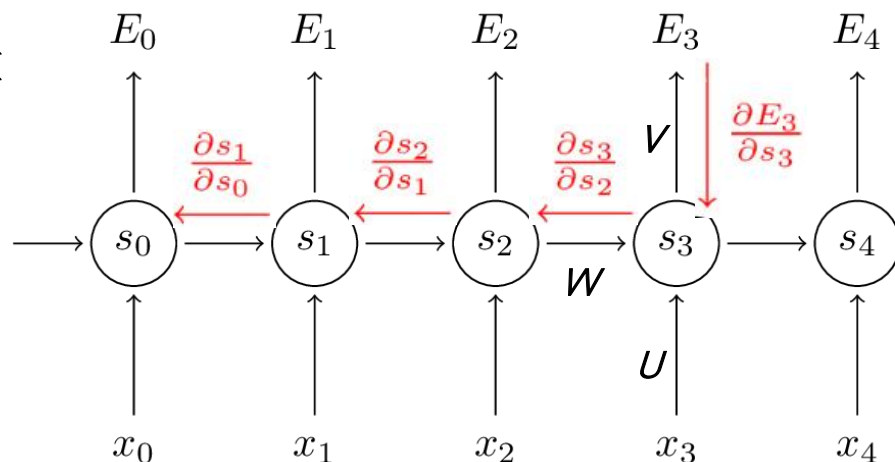
(2) 求  $E$  对于  $W$  的梯度。注意，现在情况开始变得复杂起来。

先求  $E_3$  对于  $W$  的梯度：

$$\frac{\partial E_3}{\partial W} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial W}$$

当我们求  $s_3$  对于  $W$  的偏导时。

注意到：  $s_3 = \tanh(Ux_t + Ws_2)$



其中：  $s_3$  依赖于  $s_2$ ，而  $s_2$  又依赖于  $s_1$  和  $W$ ，依赖关系一直传递到  $t = 0$  的时刻。因此，当我们计算对于  $W$  的偏导数时，不能把  $s_2$  看作是常数项！

$$\frac{\partial E_3}{\partial W} = \sum_{k=0}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial W}$$

# BPTT (backpropagation through time)

$$\frac{\partial E}{\partial U} \text{ 求和可得。}$$

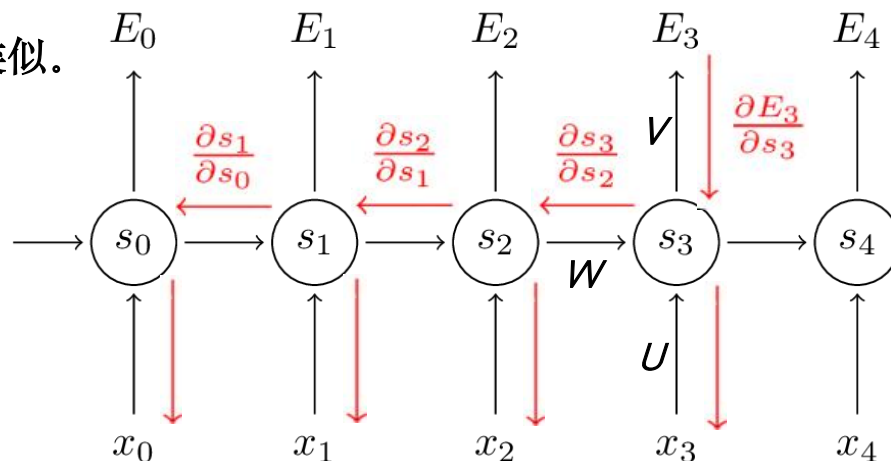
(3) 求  $E$  对于  $U$  的梯度。情况与  $W$  类似。

先求  $E_3$  对于  $U$  的梯度：

$$\frac{\partial E_3}{\partial U} = \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial U}$$

当我们求  $s_3$  对于  $W$  的偏导时。

注意到：  $s_3 = \tanh(Ux_t + Ws_2)$



同样：  $s_3$  依赖于  $s_2$ ，而  $s_2$  又依赖于  $s_1$  和  $U$ 。

类似求  $W$ ，当我们计算对于  $U$  的偏导数时，也不能把  $s_2$  看作是常数项！

$$\frac{\partial E_3}{\partial U} = \sum_{k=1}^3 \frac{\partial E_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial s_3} \frac{\partial s_3}{\partial s_k} \frac{\partial s_k}{\partial U}$$

# BPTT (backpropagation through time)

```
1: for  $t$  from  $T$  downto 1 do
2:    $do_t \leftarrow g'(o_t) \cdot dL(z_t; y_t) / dz_t$ 
3:    $db_o \leftarrow db_o + do_t$ 
4:    $dW_{oh} \leftarrow dW_{oh} + do_t h_t^\top$ 
5:    $dh_t \leftarrow dh_t + W_{oh}^\top do_t$ 
6:    $dz_t \leftarrow e'(z_t) \cdot dh_t$ 
7:    $dW_{hv} \leftarrow dW_{hv} + dz_t v_t^\top$ 
8:    $db_h \leftarrow db_h + dz_t$ 
9:    $dW_{hh} \leftarrow dW_{hh} + dz_t h_{t-1}^\top$ 
10:   $dh_{t-1} \leftarrow W_{hh}^\top dz_t$ 
11: end for
12: Return  $d\theta = [dW_{hv}, dW_{hh}, dW_{oh}, db_h, db_o, dh_0]$ .
```

参数意义:

$W_{hv}$ : 输入层到隐含层的权重参数,

$W_{hh}$ : 隐含层到隐含层的权重参数,

$W_{oh}$ : 隐含层到输出层的权重参数,

$b_h$ : 隐含层的偏移量,  $b_o$  输出层的偏移量,

$h_0$ : 起始状态的隐含层的输出, 一般初始为0.



## `min-char-rnn.py` gist: 112 lines of Python

---

字符级循环神经网络实例

初始化:

- ✓ 隐藏层向量输入 (非节点数)
  - ✓ 输入层节点个数 (每次传入网络的数据块大小)
  - ✓ 学习率
  - ✓ 输入层->隐藏层权重参数
  - ✓ 隐藏层->隐藏层权重参数
  - ✓ 隐藏层->输出层权重参数
  - ✓ 输入层->隐藏层偏置参数 (共享参数)
  - ✓ 隐藏层->输出层偏置参数 (共享参数)
-





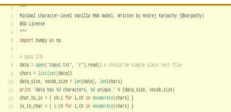
# min-char-rnn.py gist: 112 lines of Python

## 字符级循环神经网络实例

### 数据IO:

- ✓ 读文件
- ✓ 建立独有字符列表 (利用set的特性后转换为list)
- ✓ 记录数据长度&字典长度
- ✓ 建立字典到向量映射 (dict)
- ✓ 建立向量到字典映射 (dict)

min-char-rnn.py gist



Data I/O

```
1 """
2 Minimal character-level Vanilla RNN model. Written by Andrej Karpathy (@karpathy)
3 BSD License
4 """
5 import numpy as np
6
7 # data I/O
8 data = open('input.txt', 'r').read() # should be simple plain text file
9 chars = list(set(data))
10 data_size, vocab_size = len(data), len(chars)
11 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
12 char_to_ix = { ch:i for i,ch in enumerate(chars) }
13 ix_to_char = { i:ch for i,ch in enumerate(chars) }
```



# min-char-rnn.py gist: 112 lines of Python

## 字符级循环神经网络实例 初始化:

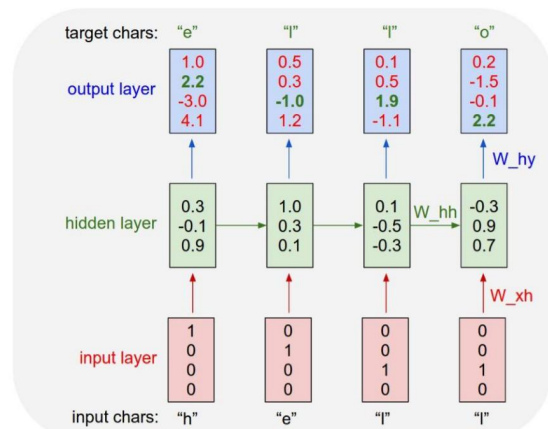
### min-char-rnn.py gist

```
1 # min-char-rnn.py
2 # min-char-rnn.py: A simple RNN model, written by Andrej Karpathy (@karpathy)
3 # MIT License
4 #
5 # Copyright (c) 2016 Andrej Karpathy
6 #
7 # Permission is hereby granted, free of charge, to any person obtaining a copy
8 # of this software and associated documentation files (the "Software"), to deal
9 # in the Software without restriction, including without limitation the rights
10 # to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
11 # copies of the Software, and to permit persons to whom the Software is
12 # furnished to do so, in connection with the Software.
```

### Initializations

```
15 # hyperparameters
16 hidden_size = 100 # size of hidden layer of neurons
17 seq_length = 25 # number of steps to unroll the RNN for
18 learning_rate = 1e-1
19
20 # model parameters
21 Wxh = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
22 Whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
23 Why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
24 bh = np.zeros((hidden_size, 1)) # hidden bias
25 by = np.zeros((vocab_size, 1)) # output bias
```

### recall:





### 主循环（跳过loss部分）：

[min-char-rnn.py](#) gist

## Main loop

```

81 n, p = 0, 0
82 mwxx, mwxxh, mwxy = np.zeros_like(Wxx), np.zeros_like(Wxxh), np.zeros_like(Wxy)
83 mbh, mby = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
84 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
85 while True:
86     # prepare inputs (we're sweeping from left to right in steps seq_length long)
87     if p+seq_length+1 >= len(data) or n == 0:
88         hprev = np.zeros((hidden_size,1)) # reset RNN memory
89         p = 0 # go from start of data
90     inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
91     targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
92
93     # sample from the model now and then
94     if n % 100 == 0:
95         sample_ix = sample(hprev, inputs[0], 200)
96         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
97         print '----\n %s \n----' % (txt, )
98
99     # forward seq_length characters through the net and fetch gradient
100     loss, dwxx, dwxxh, dwxy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
101     smooth_loss = smooth_loss * 0.999 + loss * 0.001
102     if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
103
104     # perform parameter update with Adagrad
105     for param, dparam, mem in zip([Wxx, Wxxh, Wxy, bh, by],
106                                   [dwxx, dwxxh, dwxy, dbh, dby],
107                                   [mwxx, mwxxh, mwxy, mbh, mby]):
108         mem += dparam * dparam
109         param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
110
111     p += seq_length # move data pointer
112     n += 1 # iteration counter

```



## min-char-rnn.py gist: 112 lines of Python

字符级循环神经网络实例

主循环（跳过loss部分）：

```
# prepare inputs (we're sweeping from left to right in steps seq_length long)
if p+seq_length+1 >= len(data) or n == 0:
    hprev = np.zeros((hidden_size,1)) # reset RNN memory
    p = 0 # go from start of data
inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
```

- ✓ p相当于文件指针，每次文件结束时清除隐藏层激活数据（记忆），并从头开始；
- ✓ 设定输入数据的25个字符串（p到p+25，每次p增加25）设定目标（相当于标签）字符串，是输出的顺次加1（p+1到p+25），也就是说每次预测多一位字母。



## min-char-rnn.py gist: 112 lines of Python

字符级循环神经网络实例

主循环（跳过loss部分）：

```
# sample from the model now and then
if n % 100 == 0:
    sample_ix = sample(hprev, inputs[0], 200)
    txt = ''.join(ix_to_char[ix] for ix in sample_ix)
    print '----\n %s \n----' % (txt, )
```

✓ 每100轮训练，从测试中读取数据使用RNN预测，并打印结果；





## min-char-rnn.py gist: 112 lines of Python

字符级循环神经网络实例

主循环（跳过loss部分）：

```
# forward seq_length characters through the net and fetch gradient
loss, dWxh, dWhh, dWhy, dbh, dby, hprev = lossFun(inputs, targets, hprev)
smooth_loss = smooth_loss * 0.999 + loss * 0.001
if n % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
```

✓ 损失函数以及梯度计算；

```
# perform parameter update with Adagrad
for param, dparam, mem in zip([Wxh, Whh, Why, bh, by],
                               [dWxh, dWhh, dWhy, dbh, dby],
                               [mWxh, mWhh, mWhy, mbh, mby]):
    mem += dparam * dparam
    param += -learning_rate * dparam / np.sqrt(mem + 1e-8) # adagrad update
```

✓ 更新参数，使用的是adagrad法



## Loss function

- forward pass (compute loss)
- backward pass (compute param gradient)

[illegible]

```

27 def lossFun(inputs, targets, hprev):
28     """
29     inputs, targets are both list of integers.
30     hprev is Hx1 array of initial hidden state
31     returns the loss, gradients on model parameters, and last hidden state
32     """
33     xs, hs, ys, ps = {}, {}, {}, {}
34     hs[-1] = np.copy(hprev)
35     loss = 0
36
37     # forward pass
38     for t in xrange(len(inputs)):
39         xs[t] = np.zeros((vocab_size,1)) # encode in 1-of-k representation
40         hs[t] = np.tanh(np.dot(Wxh, xs[t]) + np.dot(Whh, hs[t-1]) + bh) # hidden state
41         ys[t] = np.dot(Why, hs[t]) + by # unnormalized log probabilities for next chars
42         ps[t] = np.exp(ys[t]) / np.sum(np.exp(ys[t])) # probabilities for next chars
43         loss += -np.log(ps[t][targets[t],0]) # softmax (cross-entropy loss)
44
45     # backward pass: compute gradients going backwards
46     dwhx, dwhh, dwhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
47     dbh, dby = np.zeros_like(bh), np.zeros_like(by)
48     dhnext = np.zeros_like(hs[0])
49
50     for t in reversed(xrange(len(inputs))):
51         dy = np.copy(ps[t])
52         dy[targets[t]] -= 1 # backprop into y
53         dwhy += np.dot(dy, hs[t].T)
54         dby += dy
55         dh = np.dot(Why.T, dy) + dhnext # backprop into h
56         dhraw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity
57         dbh += dhraw
58         dwhx += np.dot(dhraw, xs[t].T)
59         dwhh += np.dot(dhraw, hs[t-1].T)
60         dhnext = np.dot(Whh.T, dhraw)
61
62     for dparam in [dwhx, dwhh, dwhy, dbh, dby]:
63         np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
64
65     return loss, dwhx, dwhh, dwhy, dbh, dby, hs[len(inputs)-1]

```



向前传播过程:  $xs \{..., i:array([0...1...]), ...\}$  使用dict和onehot编码来表示输入字符的向量空间表示向前传播以及softmax分类

```

10 #
11 Minimal character level vanilla rnn model. Written by Andre Karpathy (@karpathy)
12 #
13 import numpy as np
14 #
15 # data io
16 data = open('input.txt', 'r').read() # the simple plain text file
17 data_size, vocab_size = len(data), len(set(data))
18 print('data size %d, vocab size %d' % (data_size, vocab_size))
19 # split data into train and dev sets
20 is_train = (i < 0.9 * len(data))
21 is_val = (i < 1.0 * len(data) - 1)
22 # hyperparameters
23 hidden_size = 100 # size of hidden layer of neurons
24 num_layers = 25 # number of layers to unroll the rnn
25 learning_rate = 1e-1
26 # model parameters
27 w = np.zeros((hidden_size, hidden_size, vocab_size)) * 0.01 # input to hidden
28 w_p = np.zeros((hidden_size, hidden_size, vocab_size)) * 0.01 # hidden to hidden
29 w_h = np.zeros((hidden_size, hidden_size, vocab_size)) * 0.01 # hidden to output
30 u = np.zeros((hidden_size, 1]) # hidden bias

```

## Softmax classifier





## min-char-rnn.py gist: 112 lines of Python

### 字符级循环神经网络实例

#### 反向传播过程:

- ✓ reversed(seq): 反转一个序列对象，将其元素从后向前颠倒构建成为一个新的迭代器
- ✓ ps {..., i:class, ...} 记录softmax分类，onehot编码dy [0,0,..., 1, ...] 理论值，实际上是个概率分布，dy[i]-1意思是正确类只要不到1就需要增加（减去负梯度），由于是概率分布错误分类恒正，所以不需额外处理就会被下降掉。
- ✓ 反向通过softmax层
$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$
- ✓ 反向通过线性层
$$f(x)' = 1 - (\tanh(x))^2$$
- ✓ 反向通过tanh层



# min-char-rnn.py gist: 112 lines of Python

## 字符级循环神经网络实例

### 反向传播过程:

#### min-char-rnn.py gist

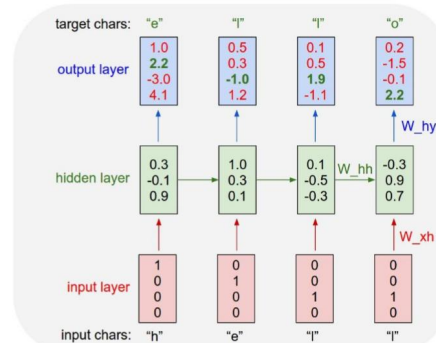
```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 # min-char-rnn.py
4 #
5 # This file is part of min-char-rnn.
6 #
7 # min-char-rnn is free software: you can redistribute it and/or modify
8 # it under the terms of the GNU General Public License as published by
9 # the Free Software Foundation, either version 3 of the License, or
10 # (at your option) any later version.
11 #
12 # min-char-rnn is distributed in the hope that it will be useful,
13 # but WITHOUT ANY WARRANTY; without even the implied warranty of
14 # MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
15 # GNU General Public License for more details.
16 #
17 # You should have received a copy of the GNU General Public License
18 # along with min-char-rnn. If not, see <http://www.gnu.org/licenses/>.
19 #
20 # Copyright (C) 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025
21 # by Andre Karpathy. All rights reserved.
```

```
22 def loss(inputs, targets, hprev):
23     """
24     Compute the loss for a batch of inputs and targets.
25     """
26     # inputs, targets are both list of integers.
27     # hprev is the hidden state from the previous batch.
28     # Returns the loss, gradients on model parameters, and last hidden state.
29     #
30     # Args:
31     #   inputs: List of integers representing input characters.
32     #   targets: List of integers representing target characters.
33     #   hprev: The hidden state from the previous batch.
34     #
35     # Returns:
36     #   loss: The loss for the batch.
37     #   dparams: Gradients on model parameters.
38     #   hlast: The last hidden state.
39     #
40     # Forward pass
41     for t in range(len(inputs)):
42         # Compute the loss for the current time step.
43         # This is done by computing the cross-entropy loss between the
44         # predicted character and the target character.
45         #
46         # Compute the hidden state for the current time step.
47         # This is done by applying the recurrent neural network
48         # operation to the previous hidden state and the current input.
49         #
50         # Compute the gradients on the model parameters.
51         # This is done by backpropagating the loss through the
52         # recurrent neural network operation.
53         #
54         # Compute the last hidden state.
55         # This is done by applying the recurrent neural network
56         # operation to the previous hidden state and the current input.
57     """
```

```
40     # Compute the loss for the current time step.
41     # This is done by computing the cross-entropy loss between the
42     # predicted character and the target character.
43     #
44     # Compute the hidden state for the current time step.
45     # This is done by applying the recurrent neural network
46     # operation to the previous hidden state and the current input.
47     #
48     # Compute the gradients on the model parameters.
49     # This is done by backpropagating the loss through the
50     # recurrent neural network operation.
51     #
52     # Compute the last hidden state.
53     # This is done by applying the recurrent neural network
54     # operation to the previous hidden state and the current input.
55     """
56     # Compute the loss for the current time step.
57     # This is done by computing the cross-entropy loss between the
58     # predicted character and the target character.
59     #
60     # Compute the hidden state for the current time step.
61     # This is done by applying the recurrent neural network
62     # operation to the previous hidden state and the current input.
63     #
64     # Compute the gradients on the model parameters.
65     # This is done by backpropagating the loss through the
66     # recurrent neural network operation.
67     #
68     # Compute the last hidden state.
69     # This is done by applying the recurrent neural network
70     # operation to the previous hidden state and the current input.
71     """
```

```
44 # backward pass: compute gradients going backwards
45 dwdx, dwhh, dwhy = np.zeros_like(Wxh), np.zeros_like(Whh), np.zeros_like(Why)
46 dbh, dby = np.zeros_like(bh), np.zeros_like(by)
47 dhnext = np.zeros_like(hs[0])
48 for t in reversed(xrange(len(inputs))):
49     dy = np.copy(ps[t])
50     dy[targets[t]] -= 1 # backprop into y
51     dwhy += np.dot(dy, hs[t].T)
52     dby += dy
53     dh = np.dot(Why.T, dy) + dhnext # backprop into h
54     dhraw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity
55     dbh += dhraw
56     dWxh += np.dot(dhraw, xs[t].T)
57     dWhh += np.dot(dhraw, hs[t-1].T)
58     dhnext = np.dot(Whh.T, dhraw)
59 for dparam in [dWxh, dWhh, dwhy, dbh, dby]:
60     np.clip(dparam, -5, 5, out=dparam) # clip to mitigate exploding gradients
61 return loss, dWxh, dWhh, dwhy, dbh, dby, hs[len(inputs)-1]
```

recall:





## 采样测试:

[min-char-rnn.py](#) gist

[illegible]

```

63 def sample(h, seed_ix, n):
64     """
65     sample a sequence of integers from the model
66     h is memory state, seed_ix is seed letter for first time step
67     """
68     x = np.zeros((vocab_size, 1))
69     x[seed_ix] = 1
70     ixes = []
71     for t in xrange(n):
72         h = np.tanh(np.dot(Wxh, x) + np.dot(Whh, h) + bh)
73         y = np.dot(Why, h) + by
74         p = np.exp(y) / np.sum(np.exp(y))
75         ix = np.random.choice(range(vocab_size), p=p.ravel())
76         x = np.zeros((vocab_size, 1))
77         x[ix] = 1
78         ixes.append(ix)
79     return ixes

```

# 深度RNN

## Multilayer RNNs

$$h_t^l = \tanh W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$h \in \mathbb{R}^n, \quad W^l [n \times 2n]$$

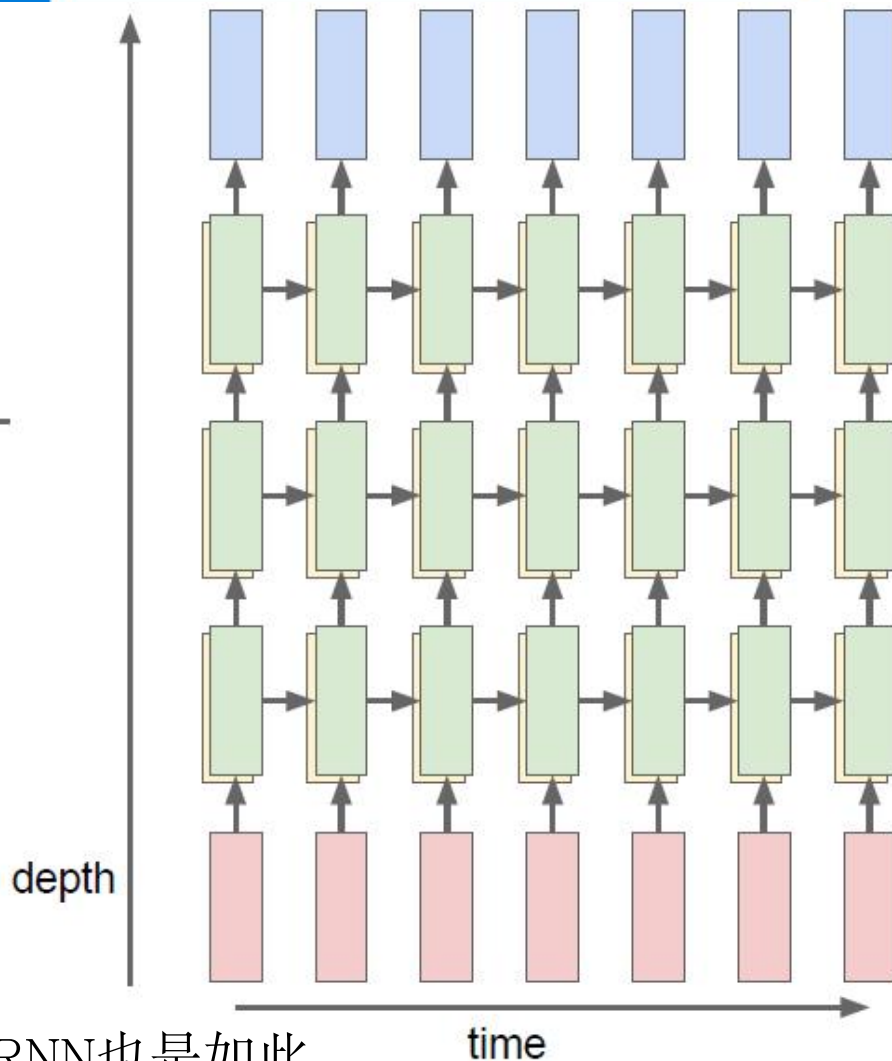
## LSTM:

$$W^l [4n \times 2n]$$

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_t^{l-1} \\ h_{t-1}^l \end{pmatrix}$$

$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$



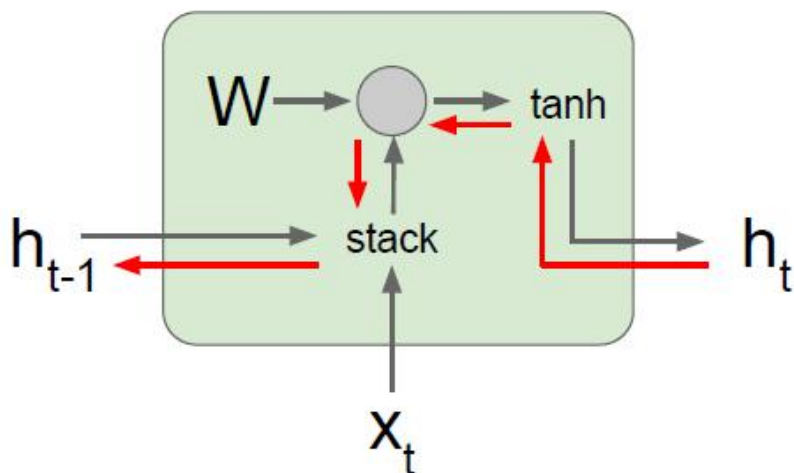
多层RNN: 当网络越深时, 性能越好, RNN也是如此。

RNN的隐藏状态可以是多层的, 通常3~4层



# Vanilla RNN 梯度流

Backpropagation from  $h_t$   
to  $h_{t-1}$  multiplies by  $W$   
(actually  $W_{hh}^T$ )



计算 $h_t$ 关于 $h_{t-1}$ 的偏微分  $\Rightarrow W^T*$

$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

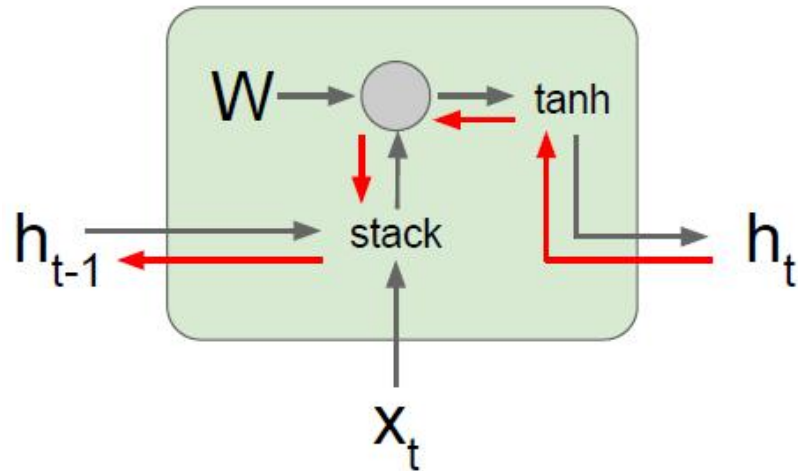
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

- ✓  $x_t$ 作为当前时间步的输入，输入前一个隐藏态 $h(t-1)$ ，两个堆叠起来与权重矩阵做乘法，得到一个输出，将输出送入tanh激活函数，得到下一个隐藏状态。
- ✓ 反向传播过程中我们会得到 $h_t$ 的导数，以及关于 $h_t$ 损失函数的导数，反向传播通过这个单元时，我们需要计算关于 $h(t-1)$ 损失函数的导数。
- ✓ 当计算反向传播时，梯度沿着红色线反向流动。梯度反向流过tanh门，反向流过矩阵乘法门，



# Vanilla RNN 梯度流

Backpropagation from  $h_t$   
to  $h_{t-1}$  multiplies by  $W$   
(actually  $W_{hh}^T$ )



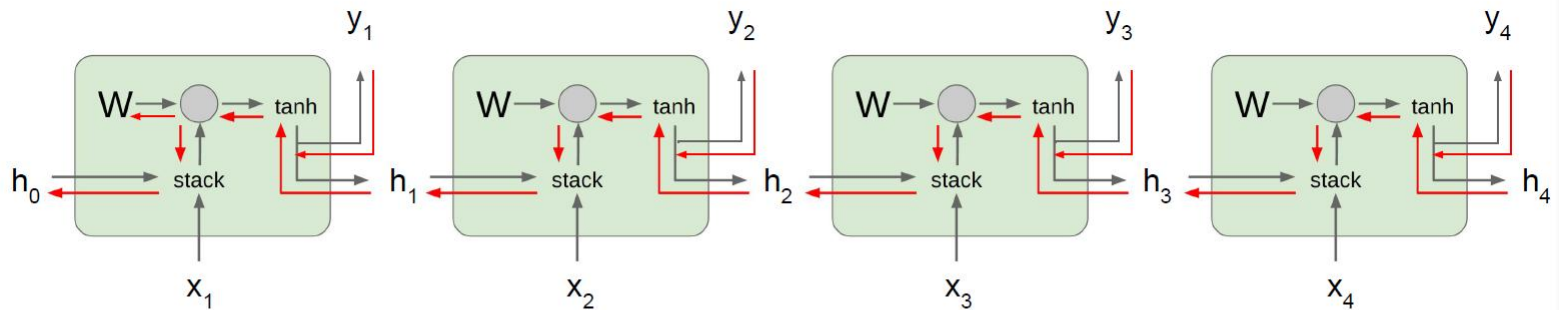
计算 $h_t$ 关于 $h_{t-1}$ 的偏微分  $\Rightarrow W^T*$

$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

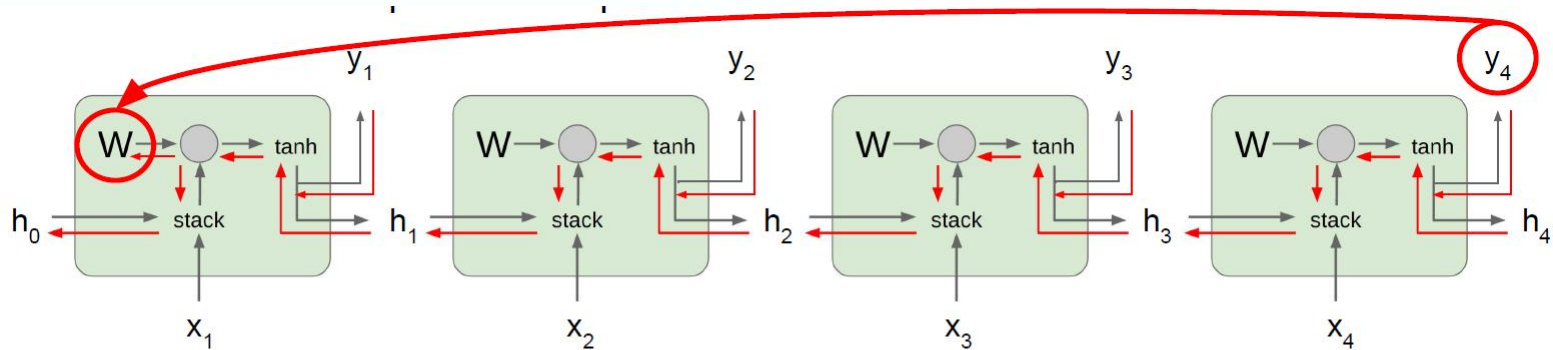


# Vanilla RNN 梯度消失和爆炸



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

# Vanilla RNN 梯度消失和爆炸

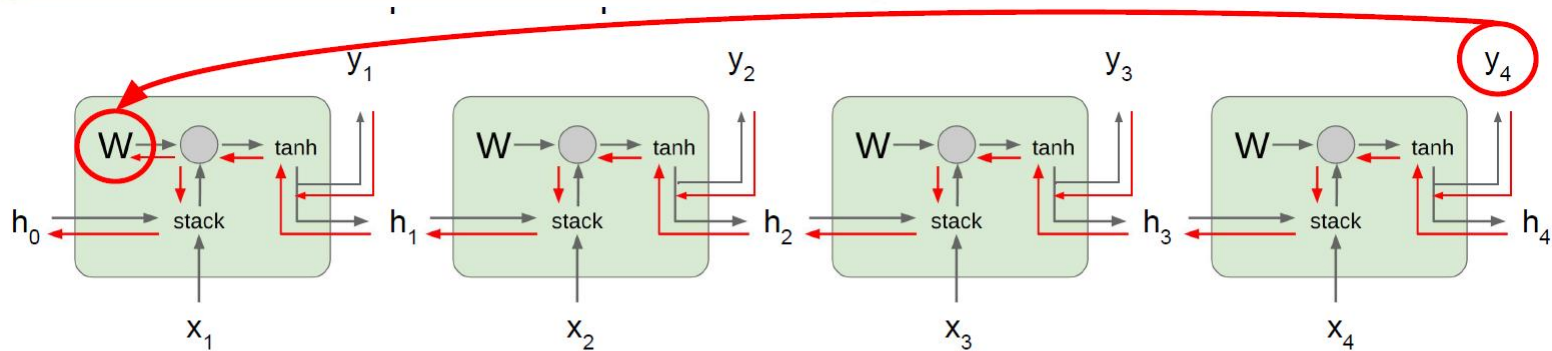


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$



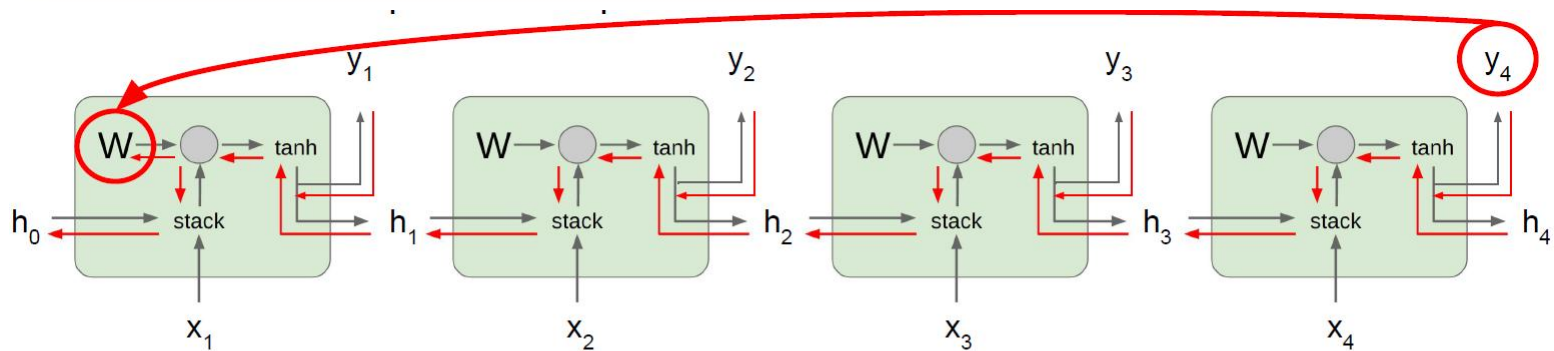
# Vanilla RNN 梯度消失和爆炸



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left( \prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

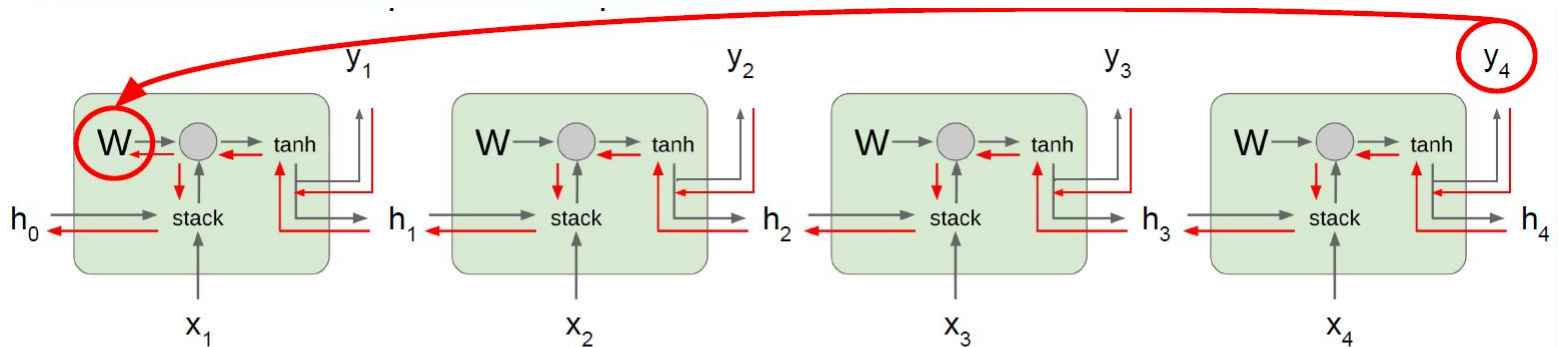
# Vanilla RNN 梯度消失和爆炸



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W} \quad \boxed{\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left( \prod_{t=2}^T \boxed{\frac{\partial h_t}{\partial h_{t-1}}} \right) \frac{\partial h_1}{\partial W}$$

# Vanilla RNN 梯度消失和爆炸



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

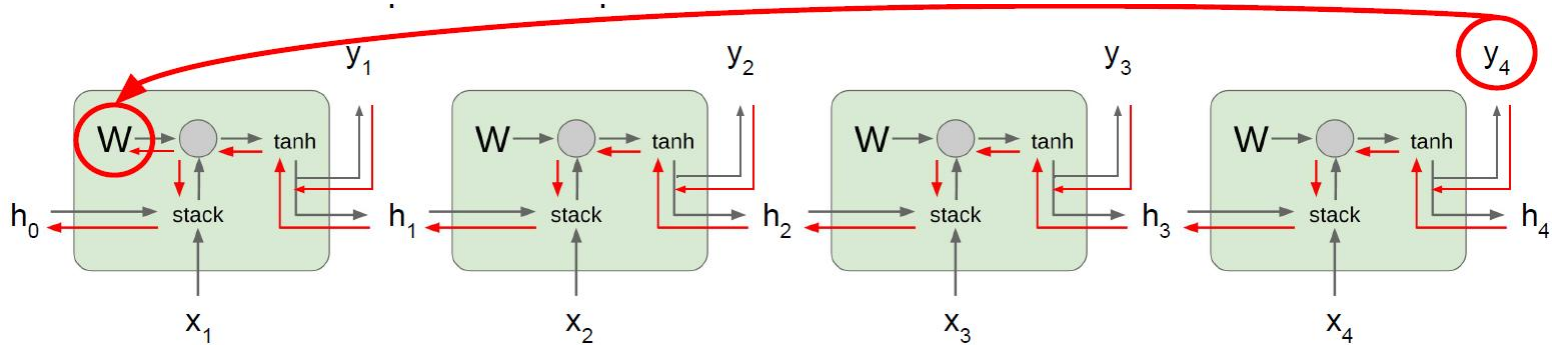
Almost always  $< 1$   
Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left( \prod_{t=2}^T \boxed{\tanh'(W_{hh} h_{t-1} + W_{xh} x_t)} \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$f(x)' = 1 - (\tanh(x))^2$$

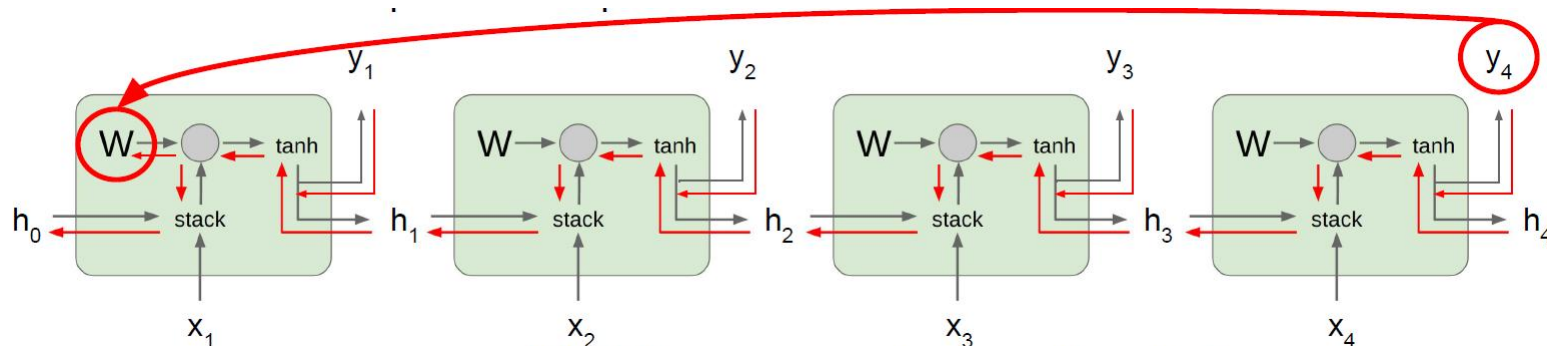
# Vanilla RNN 梯度消失和爆炸



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

What if we assumed no non-linearity?

# Vanilla RNN 梯度消失和爆炸



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

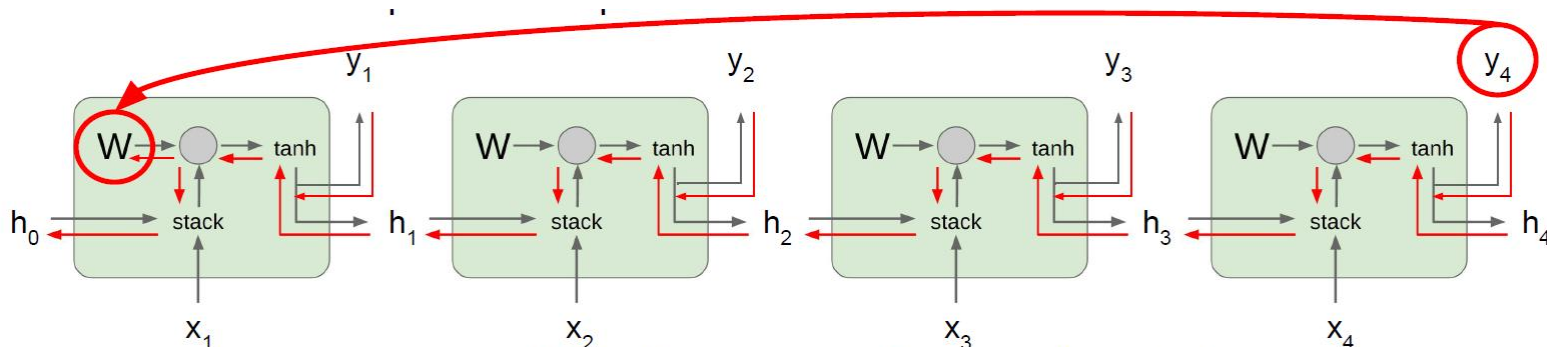
Largest singular value > 1:  
**Exploding gradients**

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{in}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1:  
**Vanishing gradients**



# Vanilla RNN 梯度消失和爆炸



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

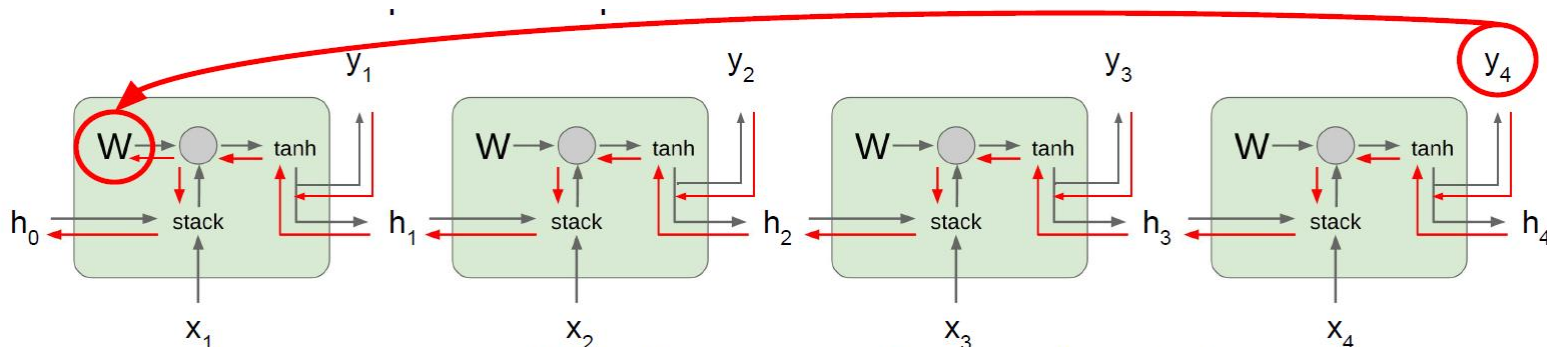
Largest singular value > 1:  
**Exploding gradients**

Largest singular value < 1:  
**Vanishing gradients**

→ **Gradient clipping:**  
Scale gradient if its  
norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

# Vanilla RNN 梯度消失和爆炸



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value > 1:  
**Exploding gradients**

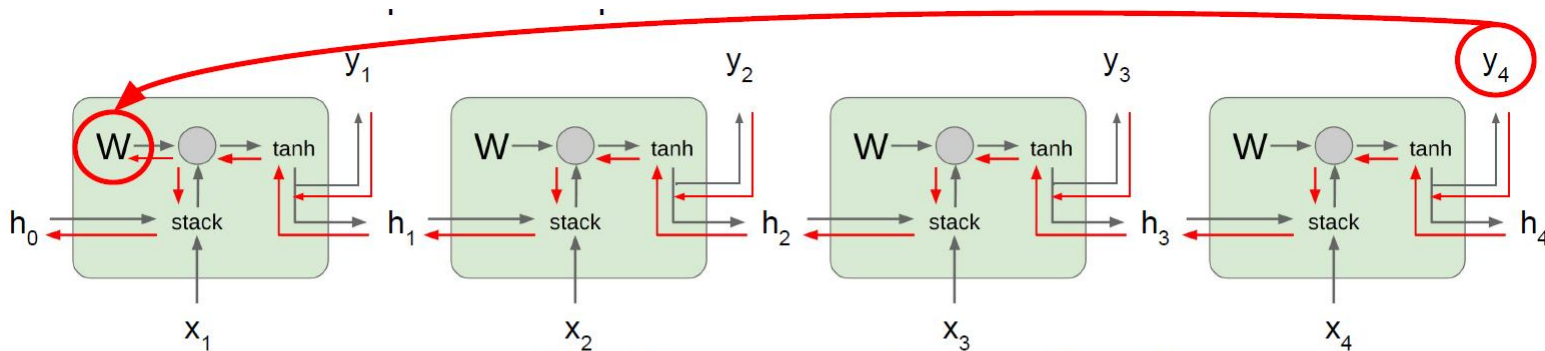
Largest singular value < 1:  
**Vanishing gradients**

**Gradient clipping:**  
Scale gradient if its  
norm is too big

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```



# Vanilla RNN 梯度消失和爆炸



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1:  
**Exploding gradients**

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{in}^{T-1}} \frac{\partial h_1}{\partial W}$$

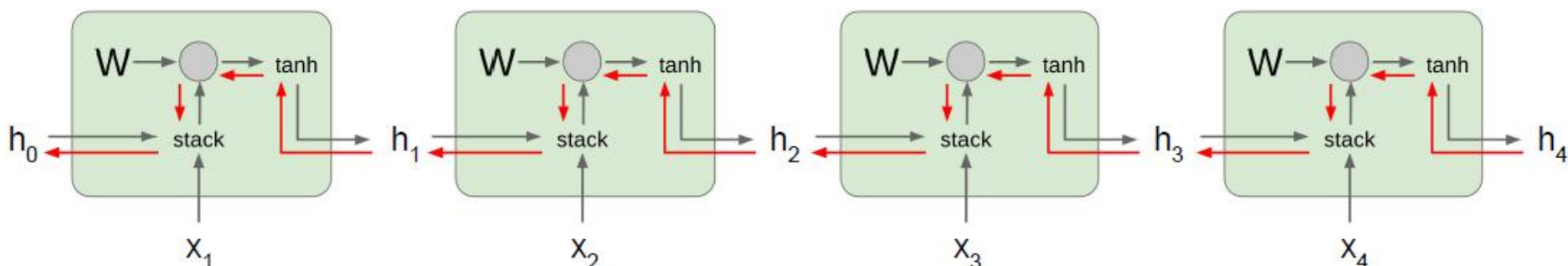
Largest singular value < 1:  
**Vanishing gradients**

→ Change RNN architecture





# Vanilla RNN 梯度消失和爆炸(总结)



传统RNN的梯度需要流经若干乘法门，导致梯度需要连乘相同的W因子

计算 $h_0$ 的梯度涉及 $W$ 的许多因素(重复 $\tanh$ )

$W$ 最大奇异值  $> 1$ :  
**Exploding gradients**

$W$ 最大奇异值  $< 1$ :  
**Vanishing gradients**

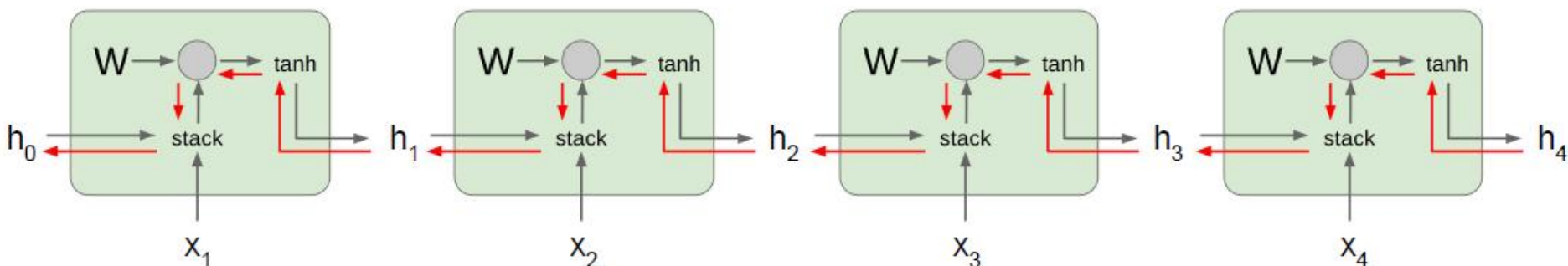
梯度截断

**Gradient clipping:**  
如果L2 norm太大的话，就缩小

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
    grad *= (threshold / grad_norm)
```

- ✓ 计算 $h_0$ 损失函数的梯度，反向传播需要经过递归神经网络中的每一个单元，每次经过一个单元都要与其中 $w$ 的转置做运算，意味着最终的表达式将会包含很多权重矩阵因子。
- ✓ 考虑标量形式，不断做乘法，会发生梯度爆炸或者梯度消失。
- ✓ 考虑矩阵时，矩阵的最大奇异值大于1，权重矩阵一个个相乘， $h_0$ 的梯度将会非常大；最大奇异值小于1，发生梯度消失。

# Vanilla RNN Gradient Flow



计算 $h_0$ 的梯度涉及 $W$ 的许多因素  
(并重复tanh)

$W$ 最大奇异值  $> 1$ :  
**Exploding gradients**

$W$ 最大奇异值  $< 1$ :  
**Vanishing gradients**

→ 改变 RNN 构架

RNN是通过隐藏态 $h$ 来表示上文信息，而梯度消失导致低时间步单元的输入值 $x$ 对高时间步的隐藏态 $h$ 影响非常小，也就是高时间步的 $h$ 几乎不含有低时间步输入 $x$ 的信息。这样模型在训练时就无法捕获到时间间隔较长的依赖关系，也就无法学习出一个能反映长期依赖关系的参数 $W$ 。



# RNN梯度

- 反向传播的长链依赖关系
  - 需要在内存中保留很多中间值
  - 蝴蝶效应
  - 梯度消失或发散
- 裁剪梯度以防止发散
$$\mathbf{g} \leftarrow \min\left(1, \frac{\theta}{\|\mathbf{g}\|}\right) \mathbf{g}$$
- 重新缩放到最大尺寸为  $\theta$  的梯度

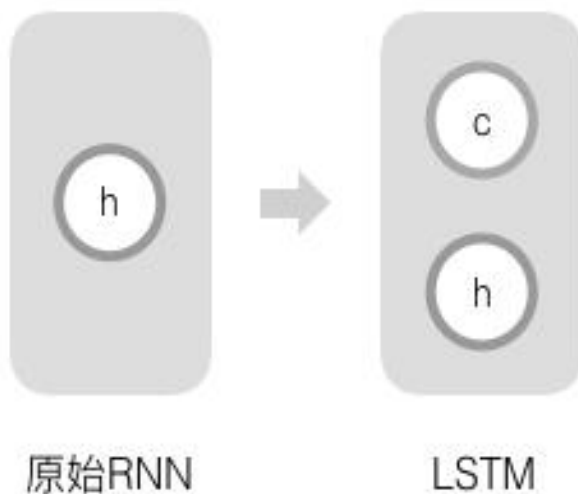
# Long Short Term Memory (LSTM)

当前预测位置和相关信息之间的间隔不断增大时，简单循环神经网络有可能会丧失学习到距离如此远的信息的能力。在这种情况下，1997年Hochreiter和Schmidhuber提出了长短期记忆网络。



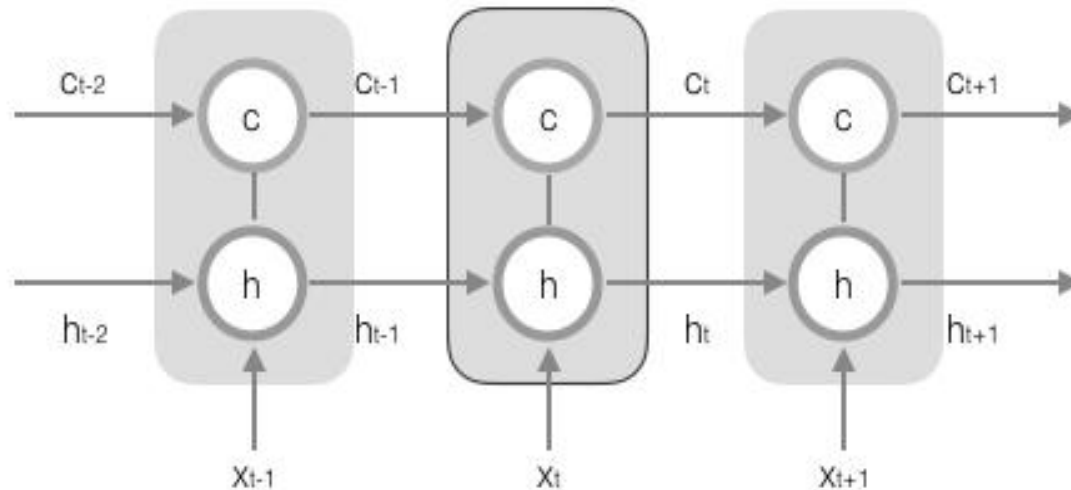
# Long Short Term Memory (LSTM)

原始RNN的隐藏层只有一个状态 $h$ ，它对于短期的输入非常敏感。LSTM增加一个状态 $c$ ，让它来保存长期的状态。



# Long Short Term Memory (LSTM)

新增加的状态 $c$ ，称为单元(细胞)状态(cell state)。把上图按照时间维度展开如下图所示。





# Long Short Term Memory (LSTM)

## Vanilla RNN

$$h_t = \tanh \left( W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

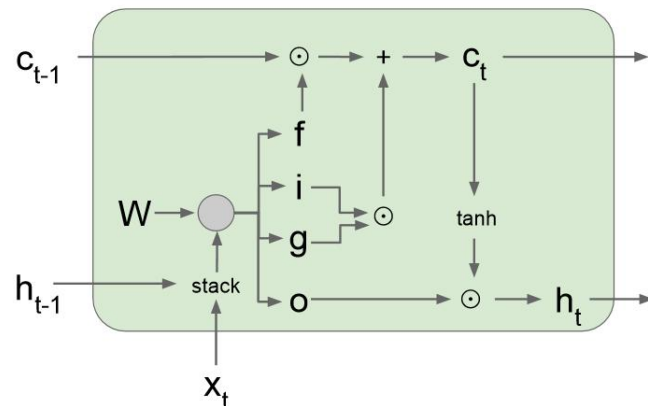
LSTM多了三个控制器: forget, input, output. forget门由  $h_{t-1}$  和  $x_t$  共同决定该状态需要遗忘哪些东西, 再与主线状态相乘达到遗忘的目的 (经过sigmoid函数输出0~1的结果); input门用来决定我们要更新什么值; output门由主线状态与  $h_{t-1}$  和  $x_t$  决定需要输出什么  $h_t$ .

## LSTM

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$



LSTM单元有隐藏状态 $h_t$ 和单元状态 $c_t$ 。单元状态的值对其他单元不可见, 只有隐藏状态可见。LSTM循环单元接收上个时间步的隐藏状态和单元状态。

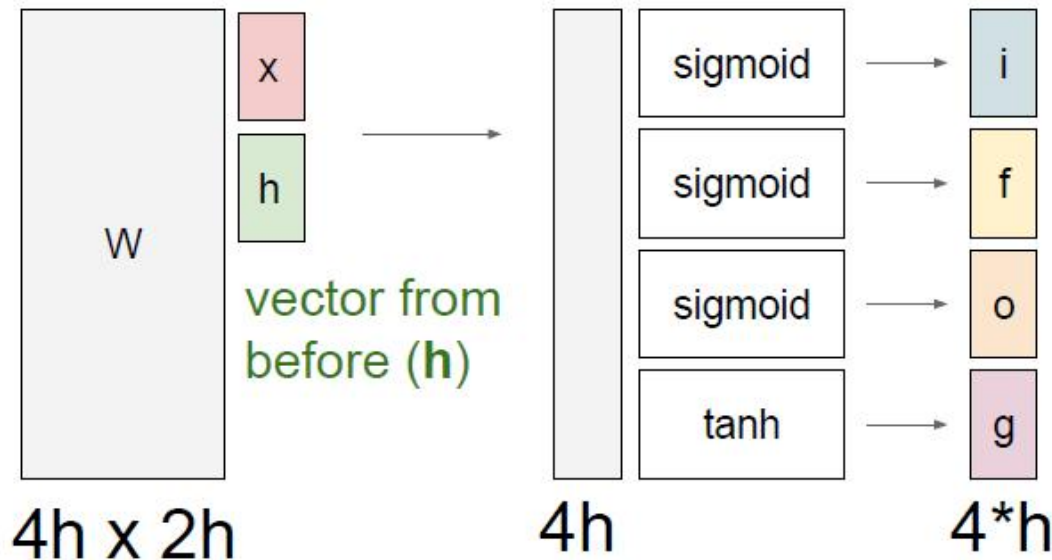


# Long Short Term Memory (LSTM)

**f**: 遗忘门, Whether to erase cell  
**i**: 输入门, whether to write to cell  
**g**: , How much to write to cell  
**o**: 输出门, How much to reveal cell

vector from  
below (**x**)

vector from  
before (**h**)



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

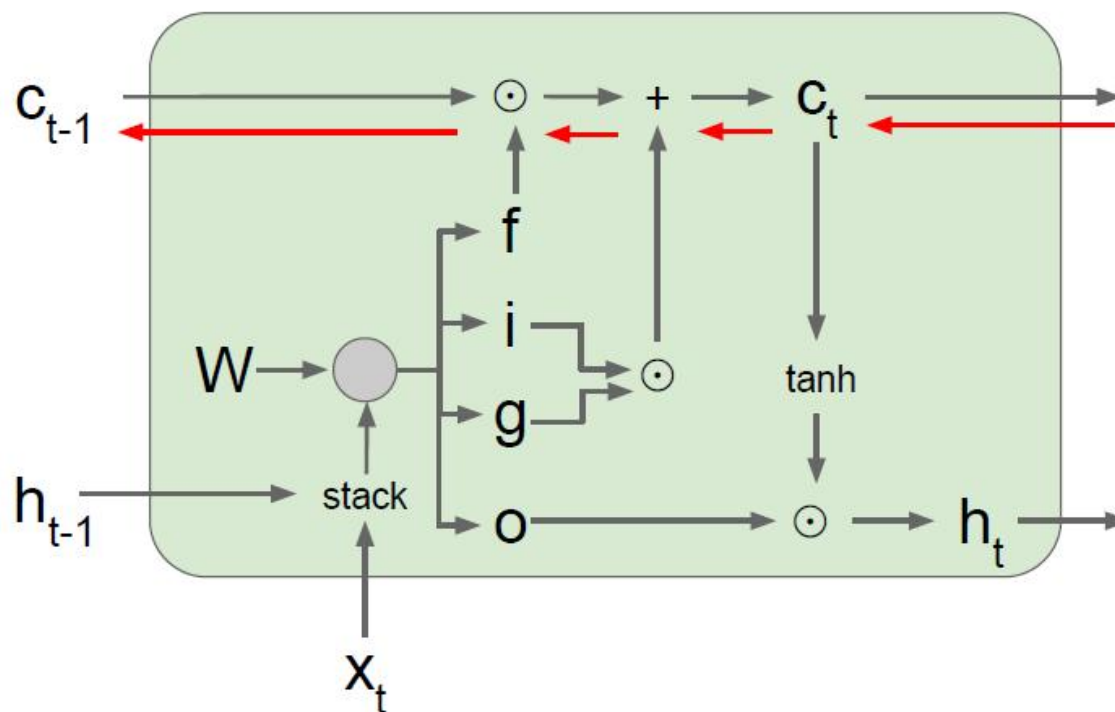
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$





# Long Short Term Memory (LSTM): 梯度流



从 $c_t$ 到 $c_{t-1}$ 的反向传播只是由 $f$ 乘以元素，没有矩阵乘以 $W$ 。

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

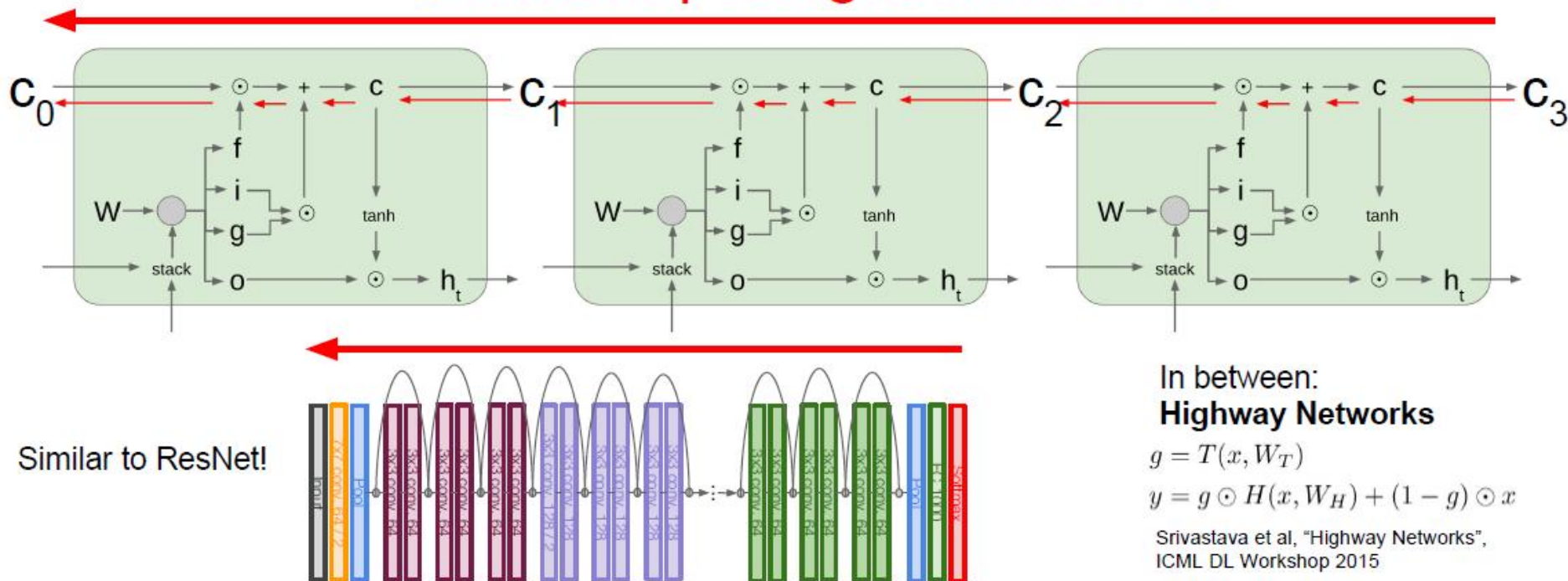
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

$c_t$ 表示单元状态，每次 $c_t$ 更新时，都要经过遗忘和写入两个阶段。遗忘阶段 $c_{t-1}$ 会与 $f$ 逐元素相乘，选择性的丢弃一些信息， $f$ 中为1的元素对应的 $c_{t-1}$ 信息得以完全保留，为0的元素对应的 $c_{t-1}$ 信息会被完全丢弃。然后， $i$ 与 $g$ 逐元素相乘，表示有多少新的信息要写入到单元状态中。之后更新 $c_t$ 。

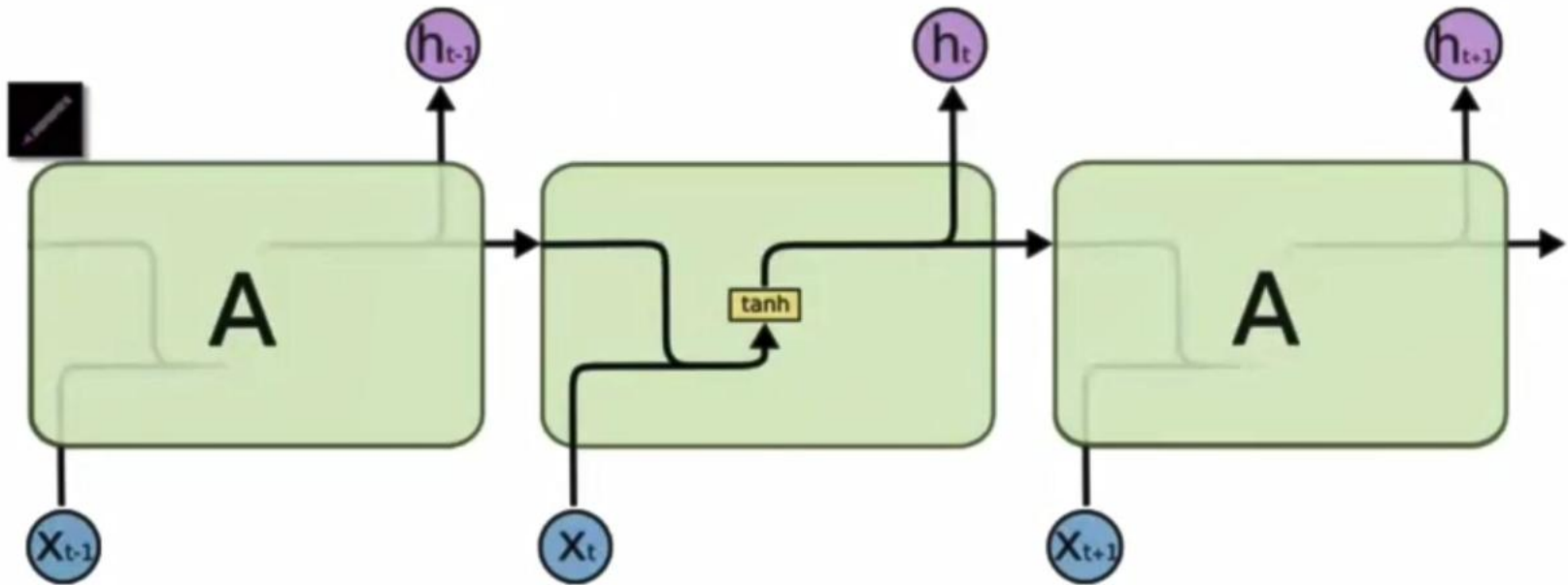
# Long Short Term Memory (LSTM): 梯度流

Uninterrupted gradient flow!



虽然流经 $h$ 的梯度流仍然会很大程度衰减，但由于单元状态 $C$ 的引入，形成了一条流经 $C$ 的额外的梯度高速公路。在流经 $C$ 的路线上，梯度经过加法门是无损的，经过乘法门时，需要逐元素乘以一个 $f$ 。相比于传统RNN，这里的情况好了很多，因为 $f$ 在不同的单元中都是不同的，一般不会出现都小于1的情况，此外，逐元素相乘也比矩阵相乘好得多。这样就使得梯度传播到低时间步单元时仍然不衰减，模型能够联系时间上相隔很远的序列信息来进行学习，克服上面提到的长期依赖的问题。

# Long Short Term Memory (LSTM)



长短期记忆网络 – LSTM是一种特殊的RNNs，它能够学习长期依赖。

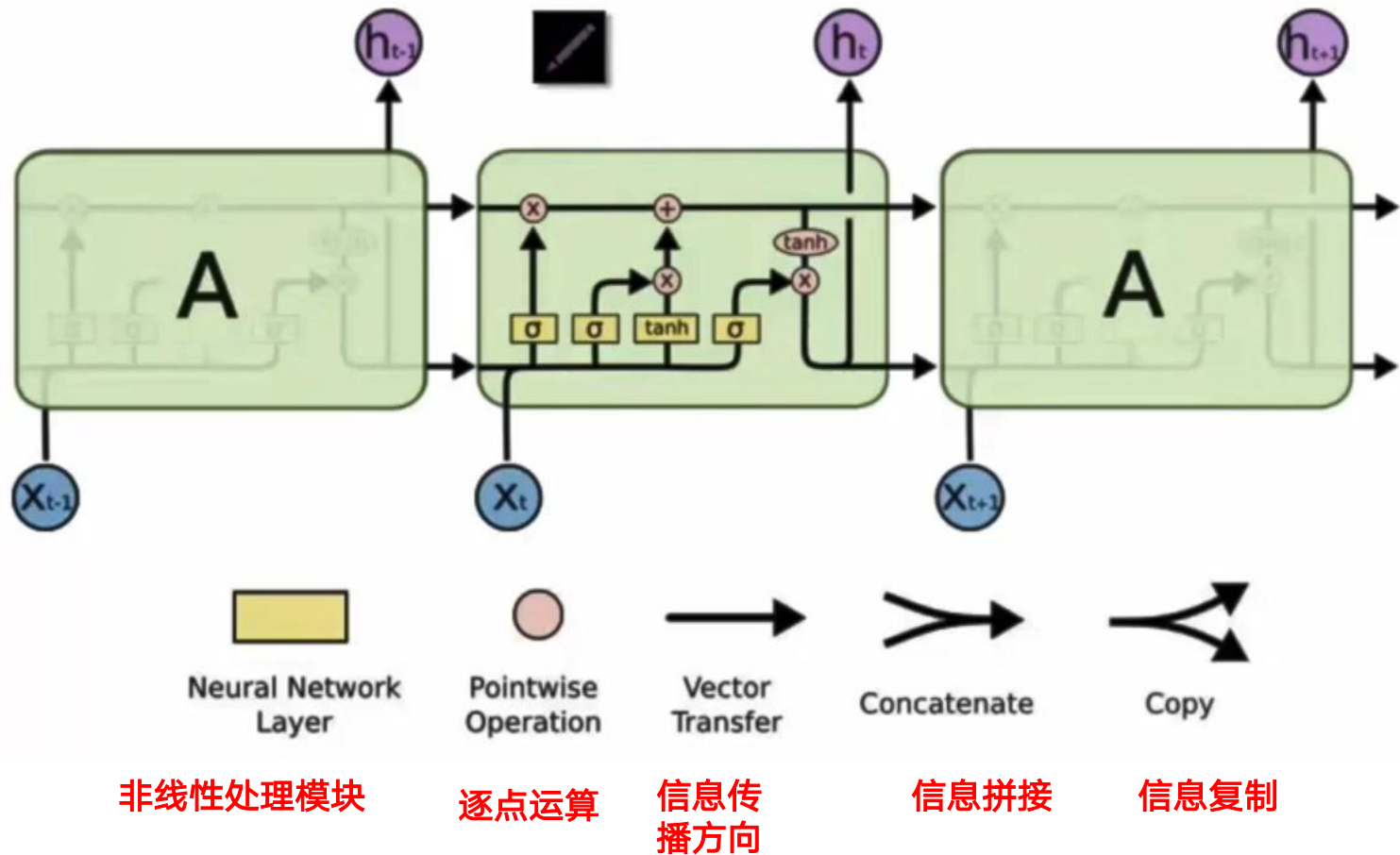


# 长短期记忆 (LSTM)

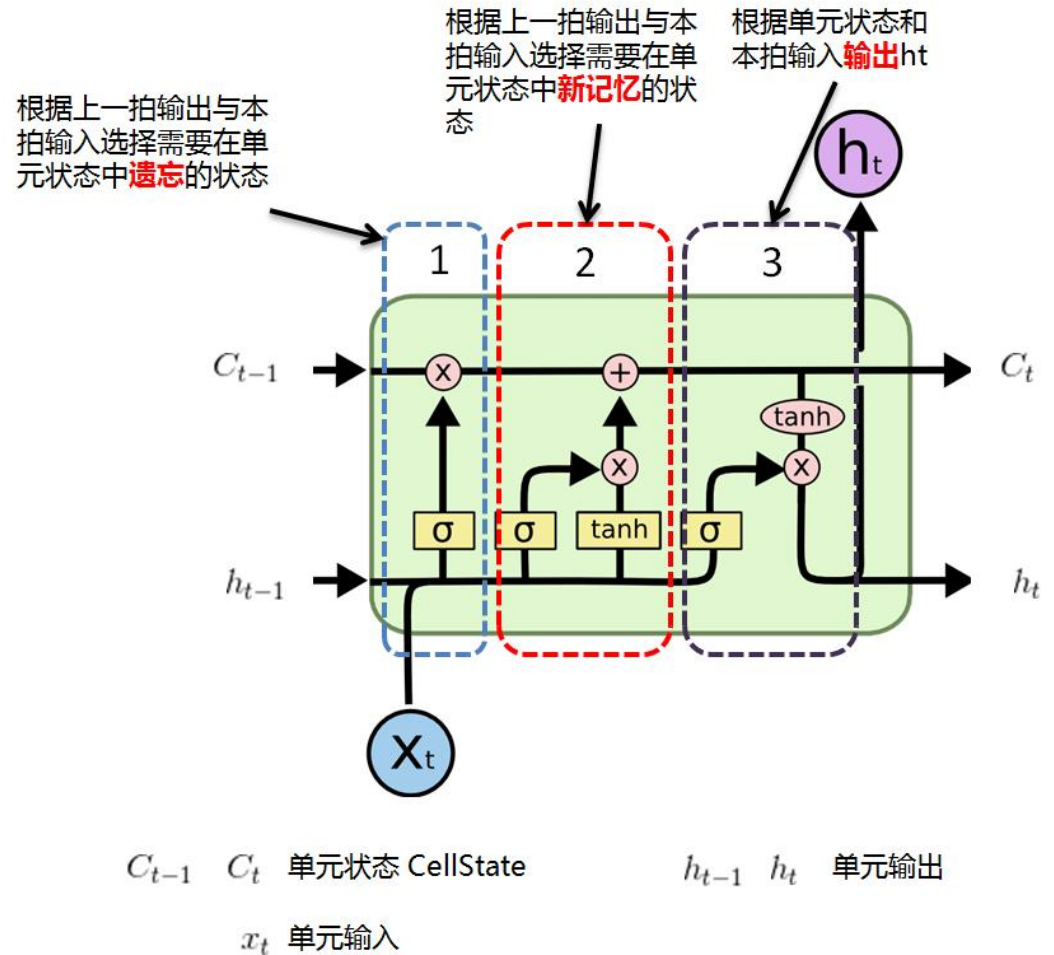
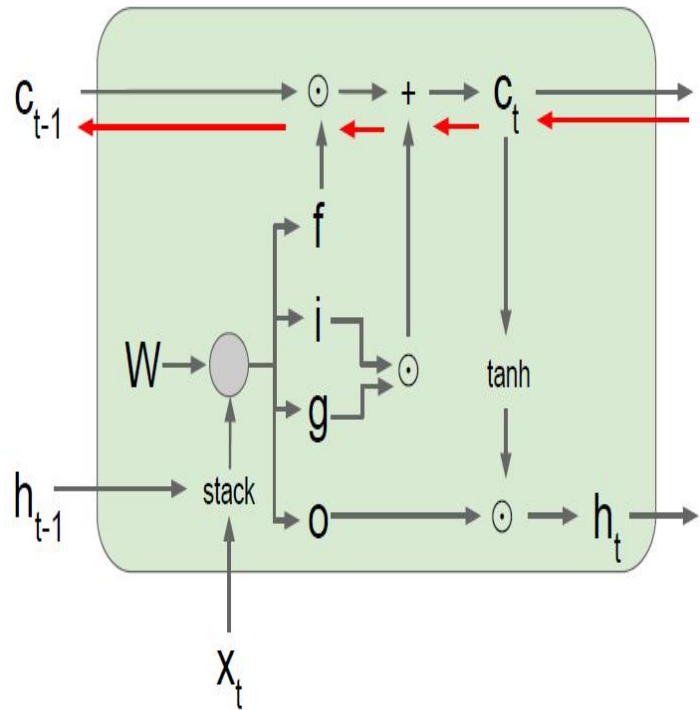
---

- 遗忘门
    - 将每个值尽可能收缩为零
  - 输入门
    - 决定是否应忽略输入数据
  - 输出门
    - 决定隐含状态是否用于 **LSTM** 生成的输出
  - 隐含状态
  - 隐含状态和记忆细胞
-

# Long Short Term Memory (LSTM)



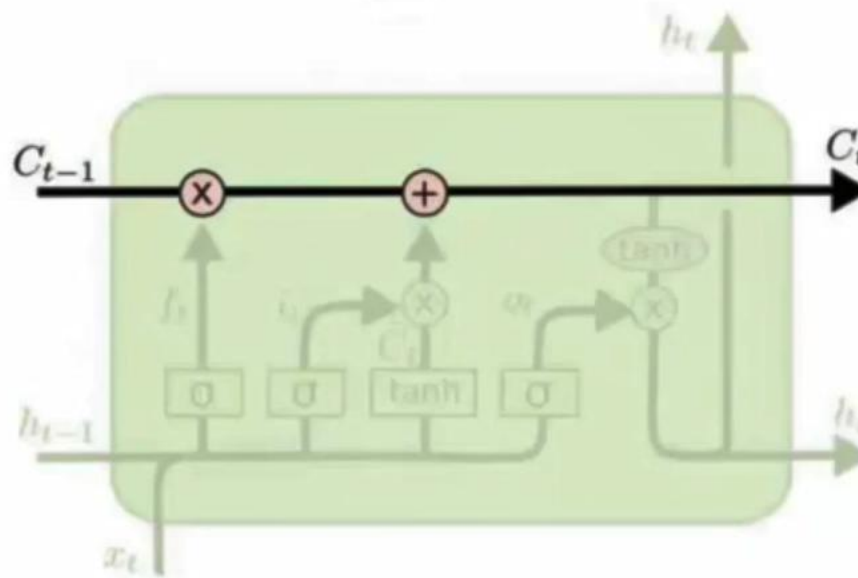
# Long Short Term Memory (LSTM)



# Long Short Term Memory (LSTM)

□ LSTM关键：“细胞状态” Cell State

□ 细胞状态类似于传送带。直接在整个链上运行，只有一些少量的线性交互。信息在上面流传保持不变会很容易。



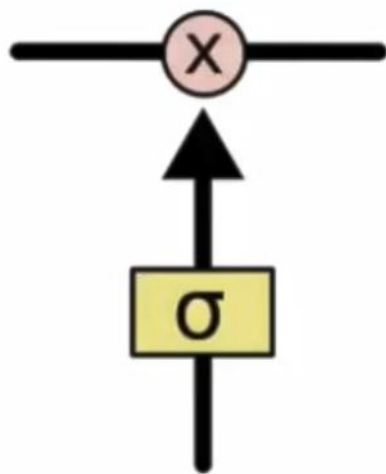
T-1时刻的记忆到现在的记忆 在传送带上往前传 发生信息的交互 可以取东西 也可以放东西上去





# Long Short Term Memory (LSTM)

- LSTM怎么控制“细胞状态”？
  - 通过“门”让信息选择性通过，来去除或者增加信息到细胞状态
  - 包含一个sigmoid神经网络层 和 一个pointwise乘法操作
  - Sigmoid 层输出0到1之间的概率值，描述每个部分有多少量可以通过。0代表“不许任何量通过”，1就指“允许任意量通过”



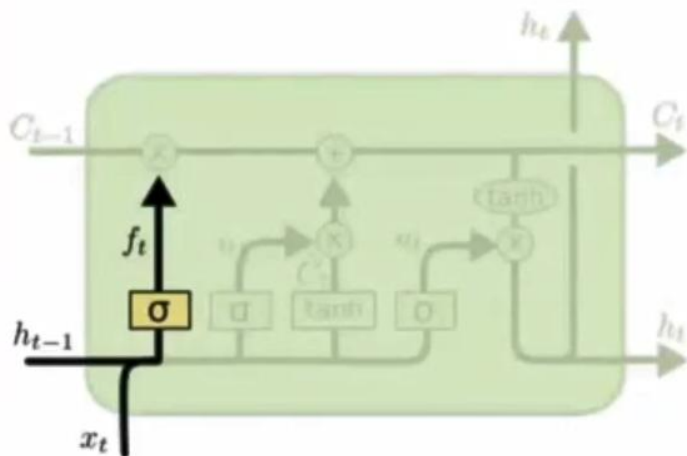
做记忆的变更：要把这部分记忆存下去



# Long Short Term Memory (LSTM)

## LSTM的几个关键“门”与操作

- 第1步：决定从“细胞状态”中丢弃什么信息 => “忘记门”
- 比如完形填空中填“他”或者“她”的问题，细胞状态可能包含当前主语的类别，当我们看到新的代词，我们希望忘记旧的代词。



上一个时刻的输出

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

以多大概率去丢掉信息      现在的输入

# Long Short Term Memory (LSTM)

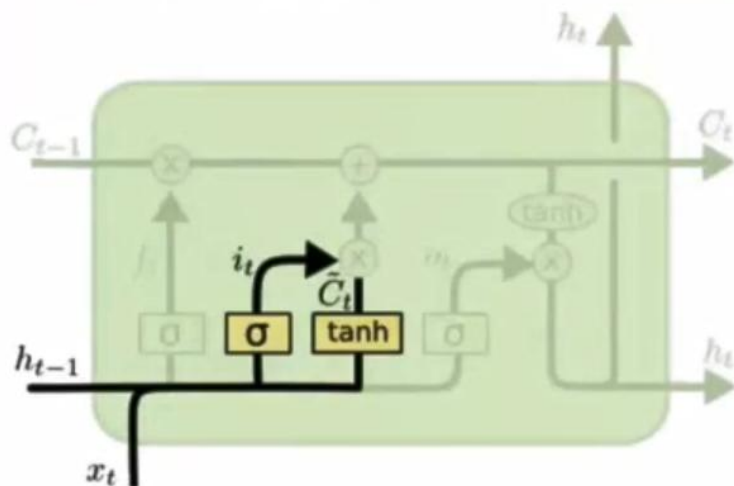
## LSTM的几个关键“门”与操作

□ 第2步：决定放什么新信息到“细胞状态”中

① Sigmoid层决定什么值需要更新 

② Tanh层创建一个新的候选值向量  $\tilde{C}_t$

③ 上述2步是为状态更新做准备



(产生一个概率值，以多少值去更新信息) 用it对Ct做过滤，哪一部分知识能够补充到我之前的知识体系中

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

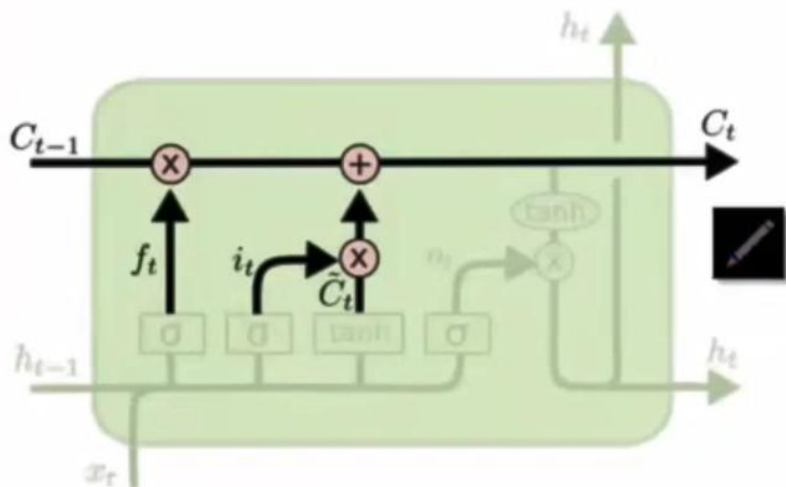
Ct:六年级这一年学到了什么

# Long Short Term Memory (LSTM)

## LSTM的几个关键“门”与操作

### □ 第3步：更新“细胞状态”

- ① 更新 $C_{t-1}$ 为 $C_t$
- ② 把旧状态与 $f_t$ 相乘，丢弃掉我们确定需要丢弃的信息
- ③ 加上 $i_t * \tilde{C}_t$ 。这就是新的候选值，根据我们决定更新每个状态的程度进行变化。



通过率 旧的记忆

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

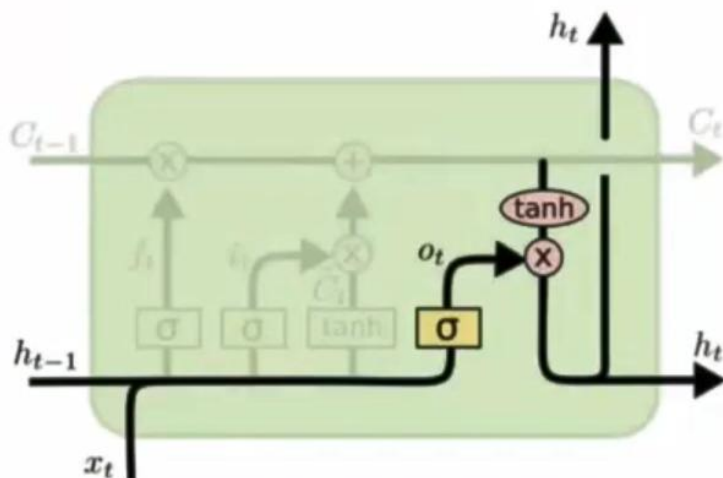
筛选器 新的知识

# Long Short Term Memory (LSTM)

## LSTM的几个关键“门”与操作

□ 第4步：基于“细胞状态”得到输出

- ① 首先运行一个sigmoid 层来确定细胞状态的哪个部分将输出
- ② 接着用tanh处理细胞状态(得到一个在-1到1之间的值)，再将它和sigmoid门的输出相乘，输出我们确定输出的那部分。
- ③ 比如我们可能需要单复数信息来确定输出“他”还是“他们”



从前六年的知识中筛出来解  
P(0,1) 决当前题目的知识

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t) \text{ 小学全六年知识}$$

解决完了给出答案

# Long Short Term Memory (LSTM)

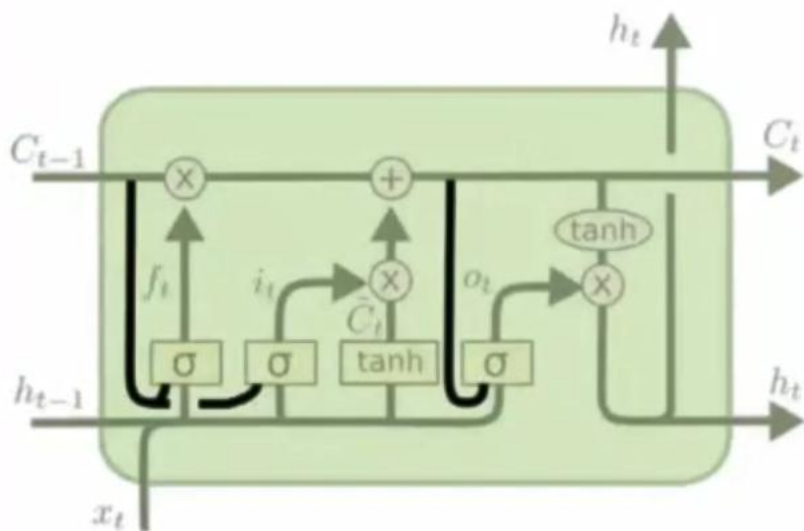
## LSTM的变种

### □ 变种1

窥视孔连接

- 增加“peephole connection”
- 让 门层 也会接受细胞状态的输入。

启发式的，没有严谨的数学推导



$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

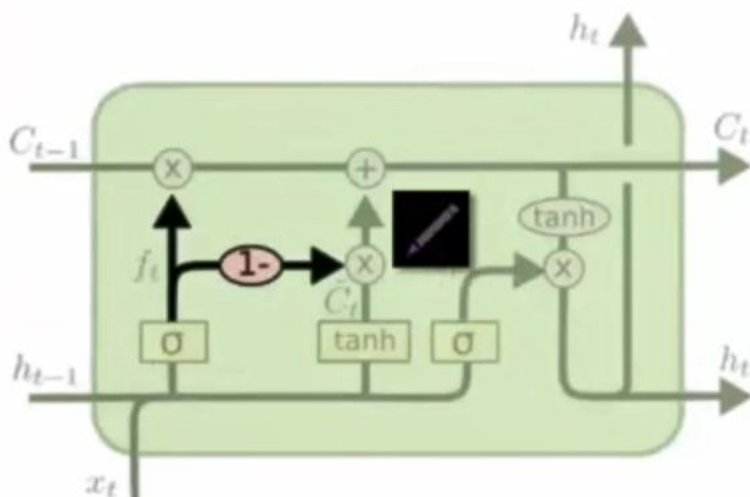
# Long Short Term Memory (LSTM)

## LSTM的变种

### □ 变种2

- 通过使用 coupled 忘记和输入门
- 之前是分开确定需要忘记和添加的信息，这里是一同做出决定。

只遗忘那些有新元素来填充的元素



忘记了0.3的信息    补充0.7的信息

$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

# 门控循环单元 (GRU)

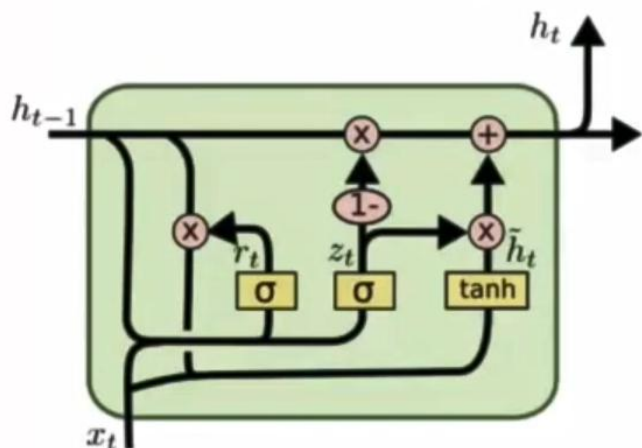




**LSTM的变种：**GRU只有两个门，分别为更新门和重置门，即图中的 $z_t$ 和 $r_t$ 。更新门用于控制前一时刻的状态信息被带入到当前状态中的程度，更新门的值越大说明前一时刻的状态信息带入越少。重置门用于控制忽略前一时刻的状态信息的程度，重置门的值越小说明忽略得越多。

### □ Gated Recurrent Unit (GRU), 2014年提出

- 将忘记门和输入门合成了一个单一的 更新门
- 同样还混合了细胞状态和隐藏状态，和其他一些改动。
- 比标准LSTM简单。



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

更新记忆

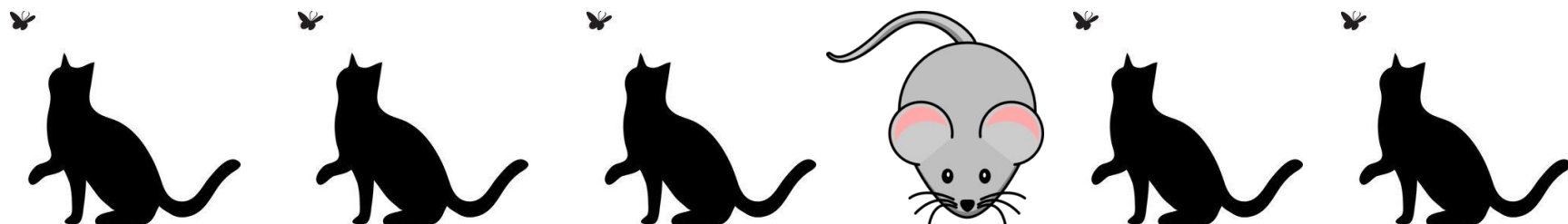
$h'$  主要包含了当前输入的  $x_t$  数据。把  $h'$  添加到当前的隐藏状态，相当于“记忆了当前时刻的状态”，类似于LSTM的选择记忆阶段。

$h_t$ : 忘记传递下来的  $h_{t-1}$  中的某些维度信息，并加入当前节点输入的某些维度信息



# 在一个序列的注意力

- 并非所有元素都具有同等意义



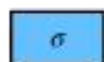
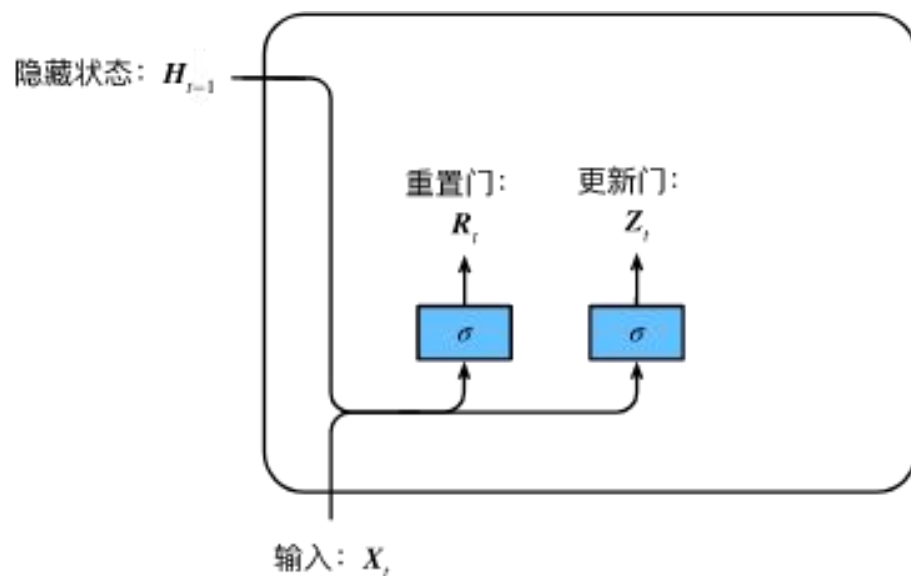
- 只记住相关的元素
  - 需要注意的机制 (更新门)
  - 需要忘记的机制 (重置门)

# 门控循环单元



$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$$

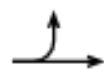
$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$



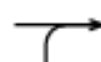
全连接层和激活函数



按元素运算符



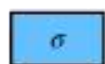
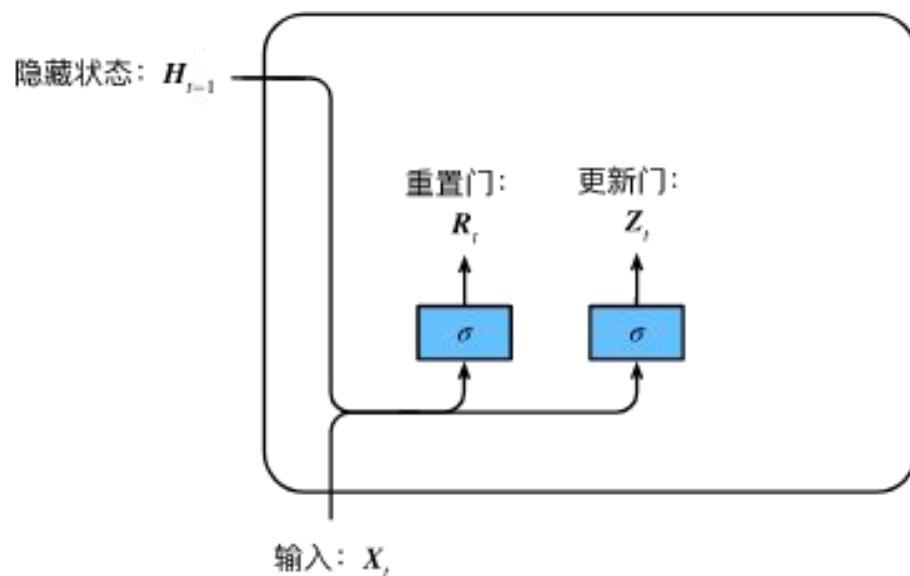
复制



连结

# 候选隐含状态

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$



全连接层和激活函数



按元素运算符



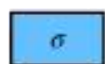
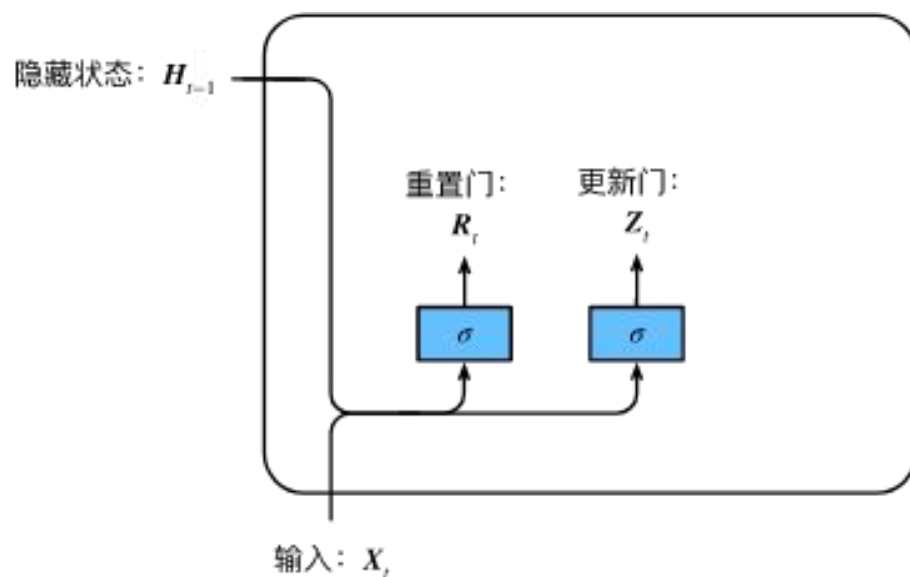
复制



连结

# 隐含状态

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$



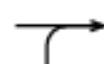
全连接层和激活函数



按元素运算符

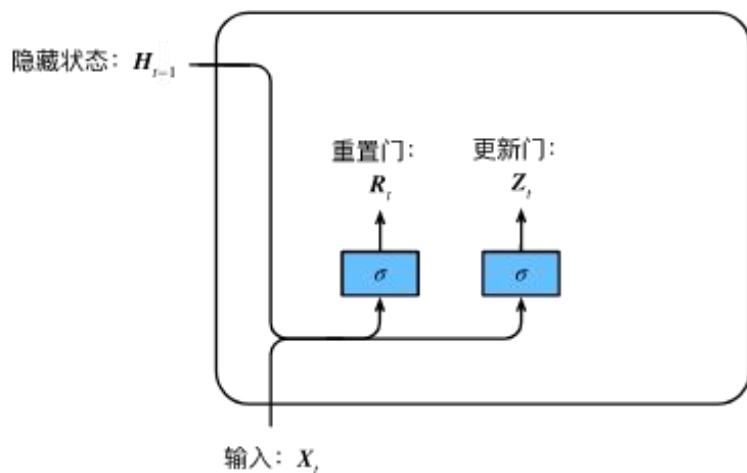


复制



连结

# 门控循环单元 (GRU) 总结



$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$



全连接层和激活函数



按元素运算符



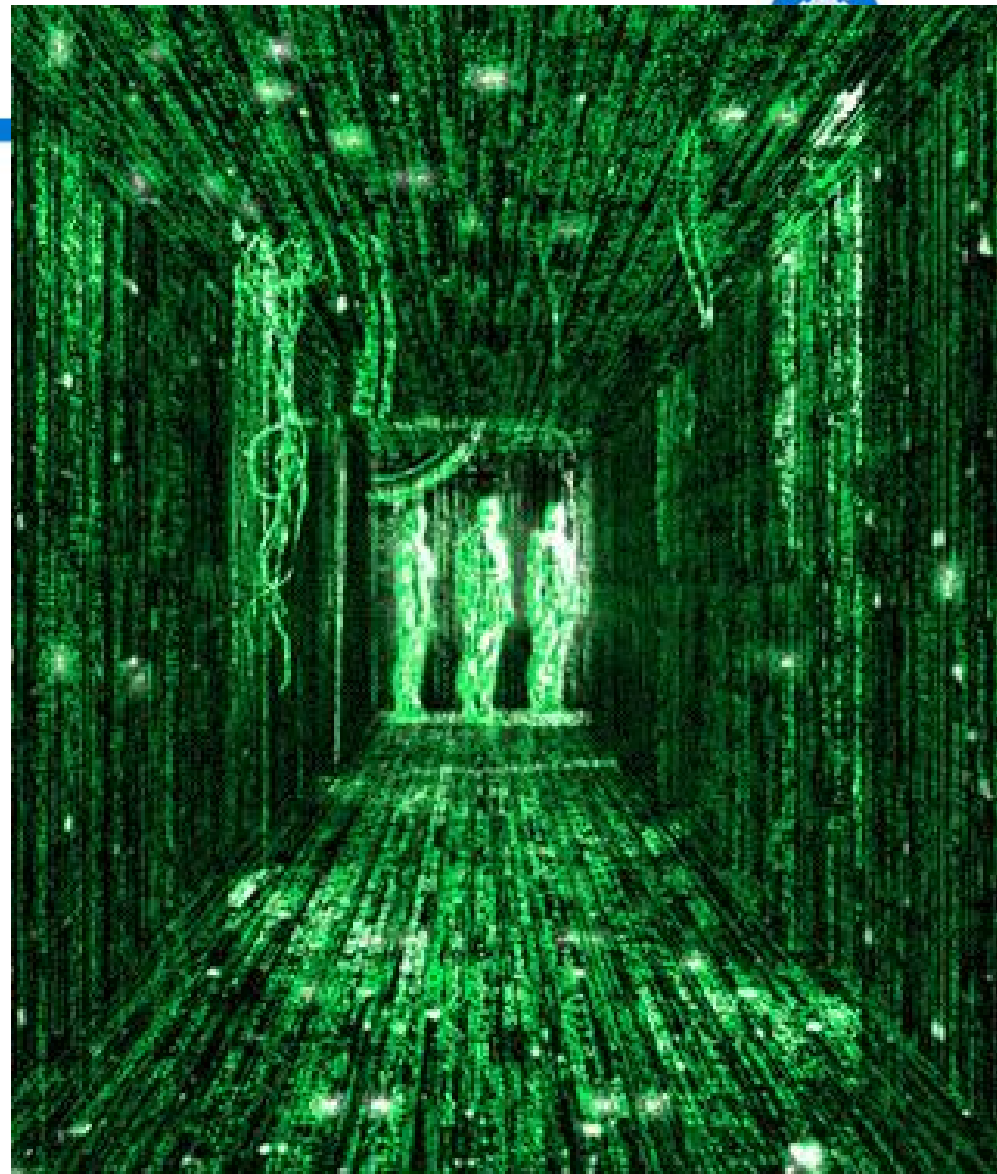
复制



连结

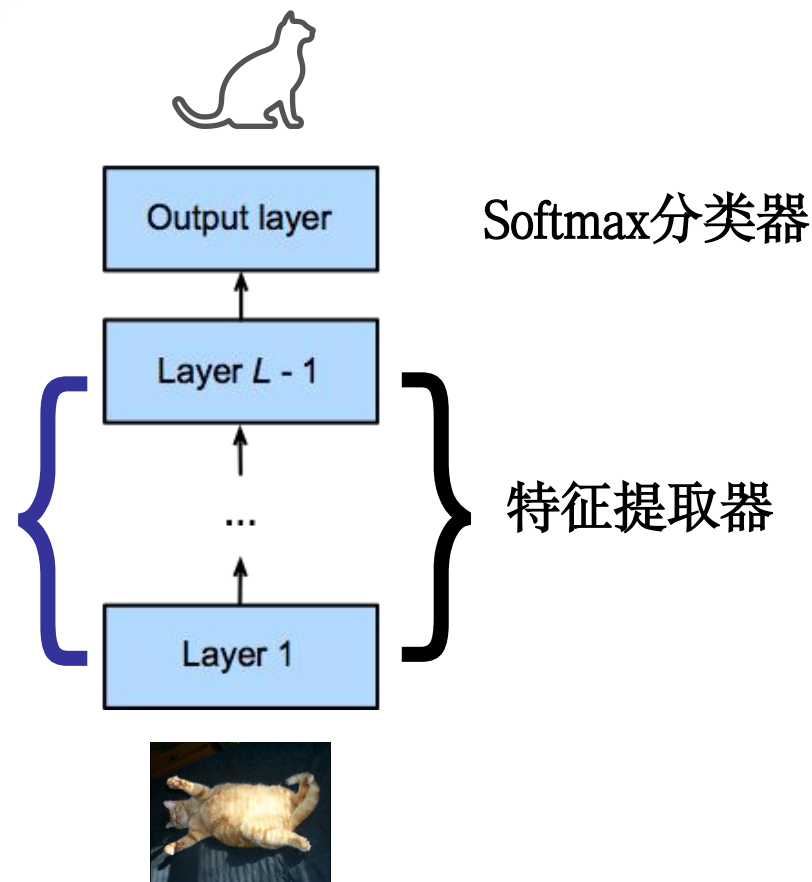
---

# 编码器 - 解码器 (Encoder - Decoder)



# 重新思考 CNN

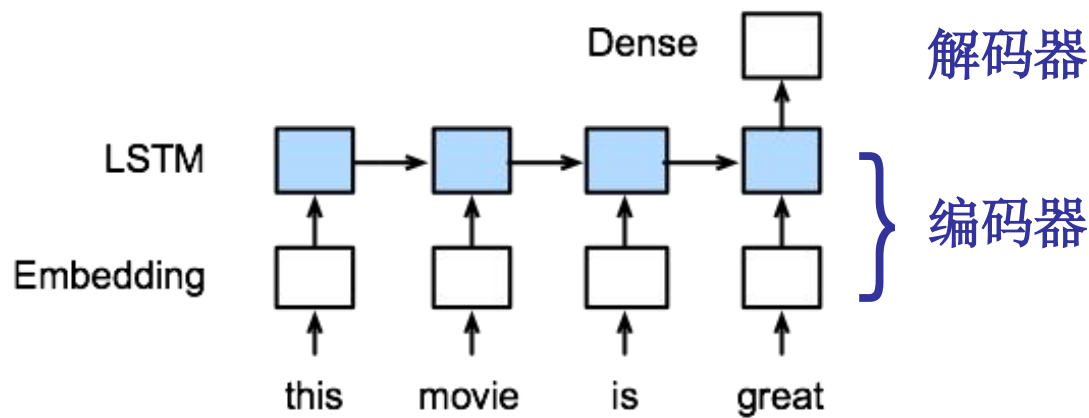
- 编码器(Encoder): 将输入编码为中间表示 (特征)  
解码器
- 解码器(Decoder): 将特征解码为输出  
编码器





# 重新思考 RNN

- 编码器 (Encoder):  
将输入编码为中间表示 (特征)
- 解码器 (Decoder):  
将特征解码为输出







某些机器学习场景中很多问题的输入和输出是不定长的序列。以机器翻译为例，输入是可以是英语的一段话，输出可以是法语的一段话，输入输出皆不定长，例如

□ 英语: They are watching.

□ 法语: Ils regardent.

当输入输出都是不定长序列时，我们可以使用编码器—解码器（encoder-decoder）或者 seq2seq。它们分别基于 2014 年的两个工作：

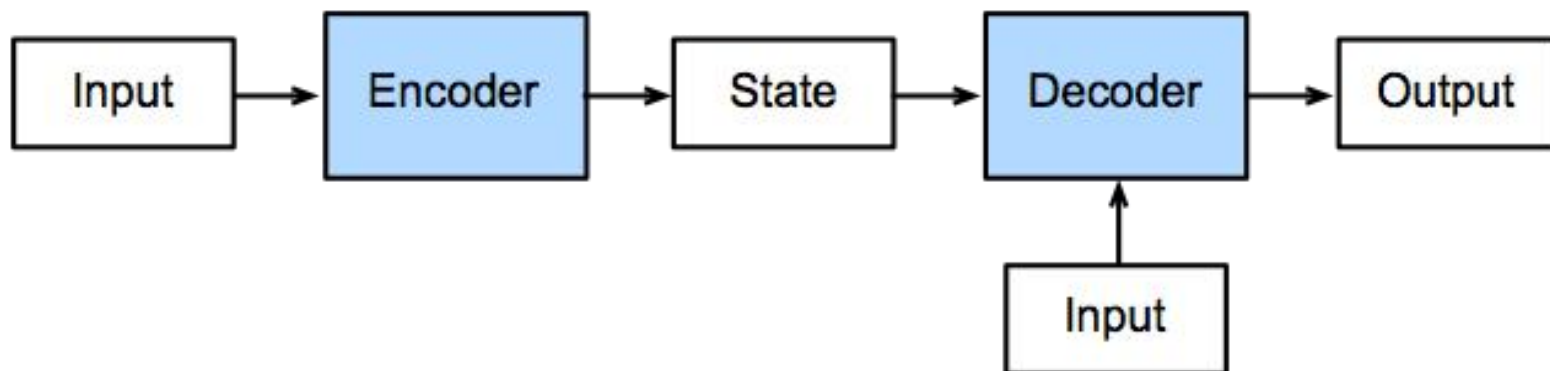
- Cho et al., Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation
- Sutskever et al., Sequence to Sequence Learning with Neural Networks

以上两个工作本质上都用到了两个循环神经网络，分别叫做编码器和解码器。编码器对应输入序列，解码器对应输出序列。



# 编码器 - 解码器架构

- 模型分为两部分
  - 编码器 (Encoder) 加工输入
  - 解码器 (Decoder) 生成输出





# 编码器 Class

---

```
class Encoder(nn.Block):  
    def __init__(self, **kwargs):  
        super(Encoder, self).__init__(**kwargs)  
  
    def forward(self, X):  
        raise NotImplementedError
```



# 解码器 Class

- 使用编码器输出和任何其他信息创建状态

```
class Decoder(nn.Block):  
    def __init__(self, **kwargs):  
        super(Decoder, self).__init__(**kwargs)  
  
    def init_state(self, enc_outputs, *args):  
        raise NotImplementedError  
  
    def forward(self, X, state):  
        raise NotImplementedError
```

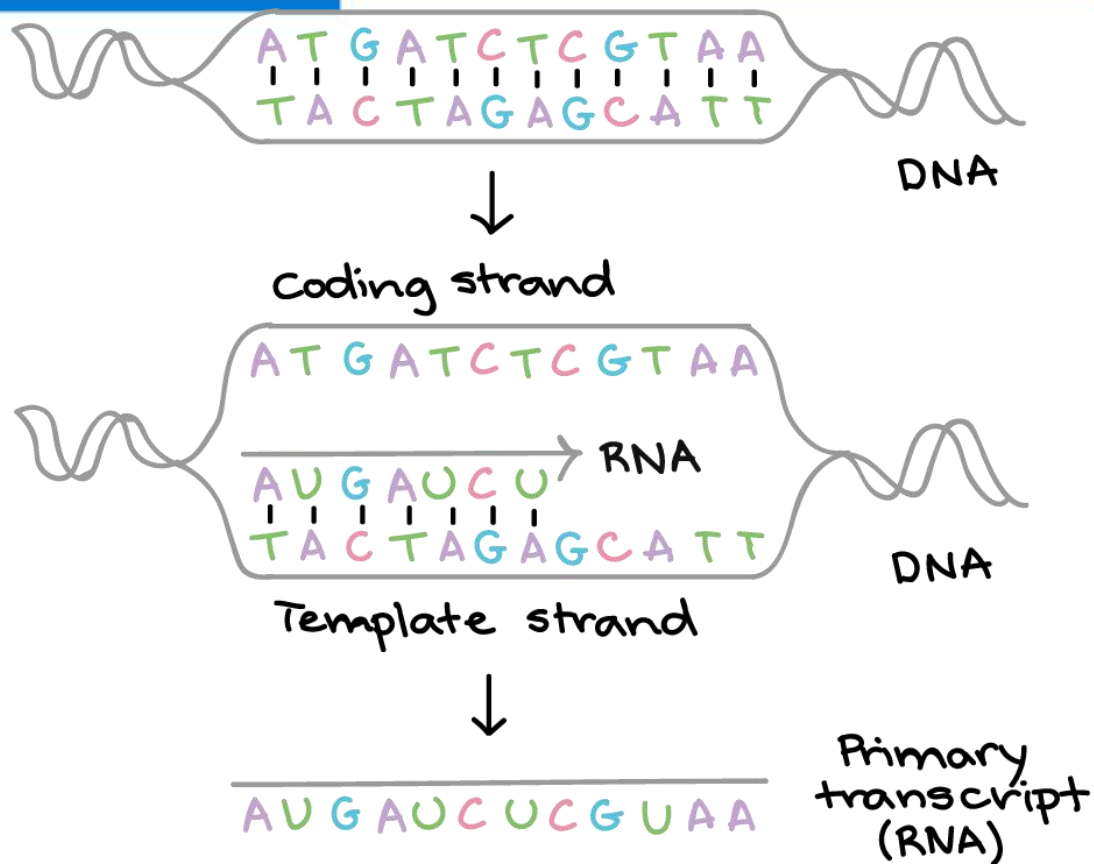


# 编码器-解码器 Class

```
class EncoderDecoder(nn.Block):
    def __init__(self, encoder, decoder, **kwargs):
        super(EncoderDecoder, self).__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, enc_X, dec_X, *args):
        enc_outputs = self.encoder(enc_X)
        dec_state = self.decoder.init_state(enc_outputs, *args)
        return self.decoder(dec_X, dec_state)
```

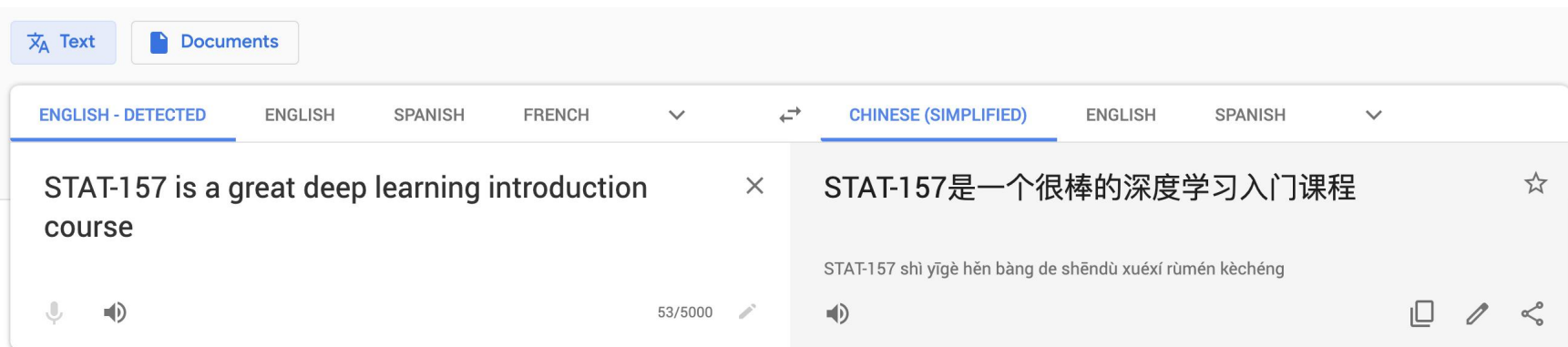
# Seq2seq 模型





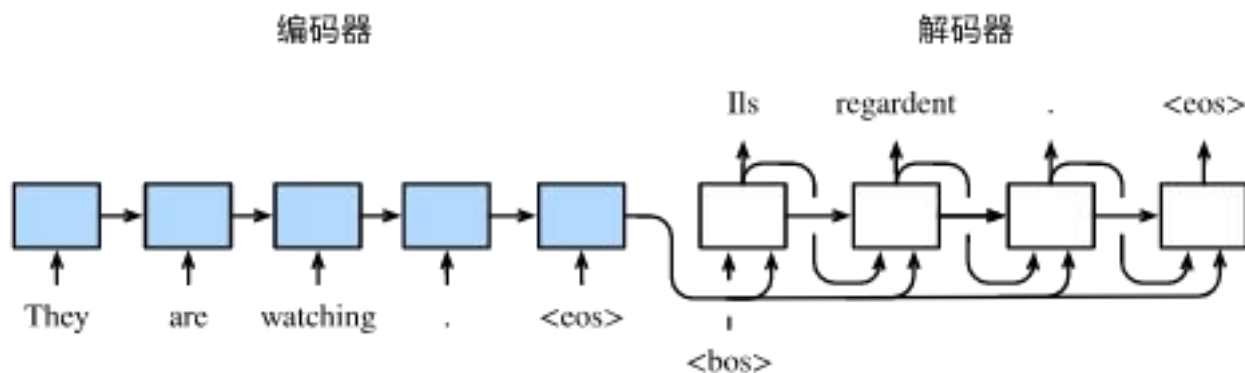
# 机器翻译

- 给定源语言中的句子，翻译成目标语言
- 这两个序列可以具有不同的长度



# Seq2seq 模型

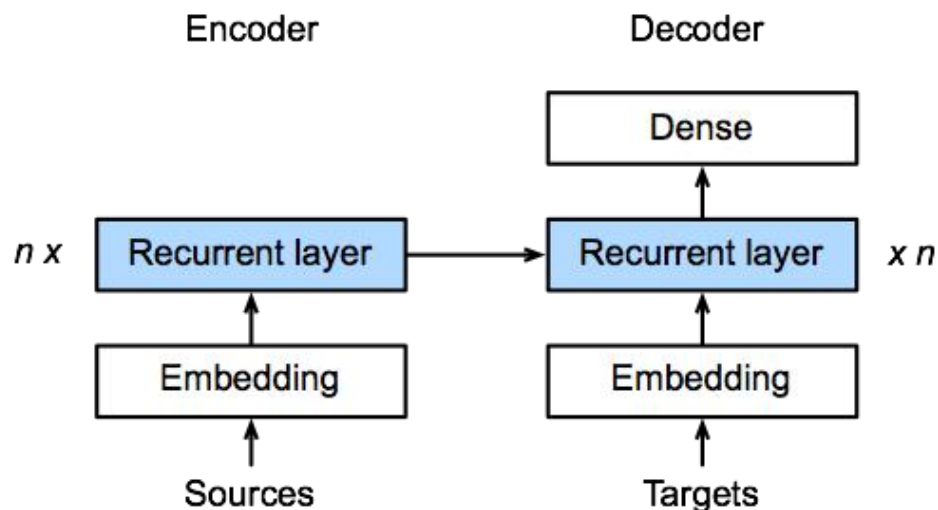
- 编码器是读取输入序列的 RNN
- 解码器使用另一个 RNN 来生成输出





# Seq2seq 模型

- 编码器是没有输出层的标准 RNN 模型
- 编码器在上一时间步骤中的隐含状态用作解码器的初始隐藏状态

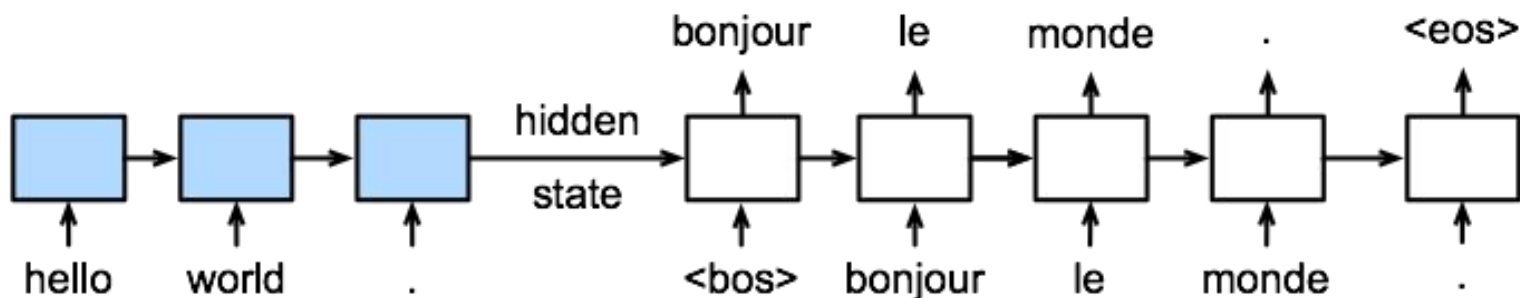


# 机器翻译训练

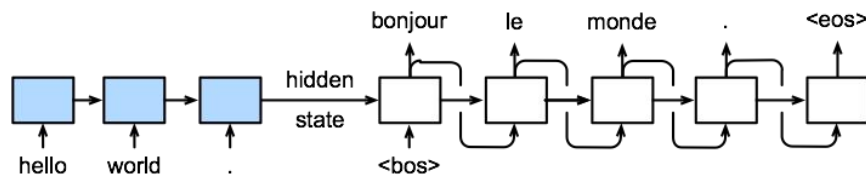
- 在训练期间，解码器 (Decoder) 用目标语言句子作为输入

Encoder

Decoder



Predict:



# 总结

- RNN在架构设计方面有很大的灵活性
- Vanilla RNNs很简单，但不工作得很好
- 使用LSTM或GRU的共同点：它们的加性相互作用改善梯度流动
- RNN中梯度的后向流动可能激增或消失。

激增由渐变剪辑控制。消失是由加性相互作用（LSTM）